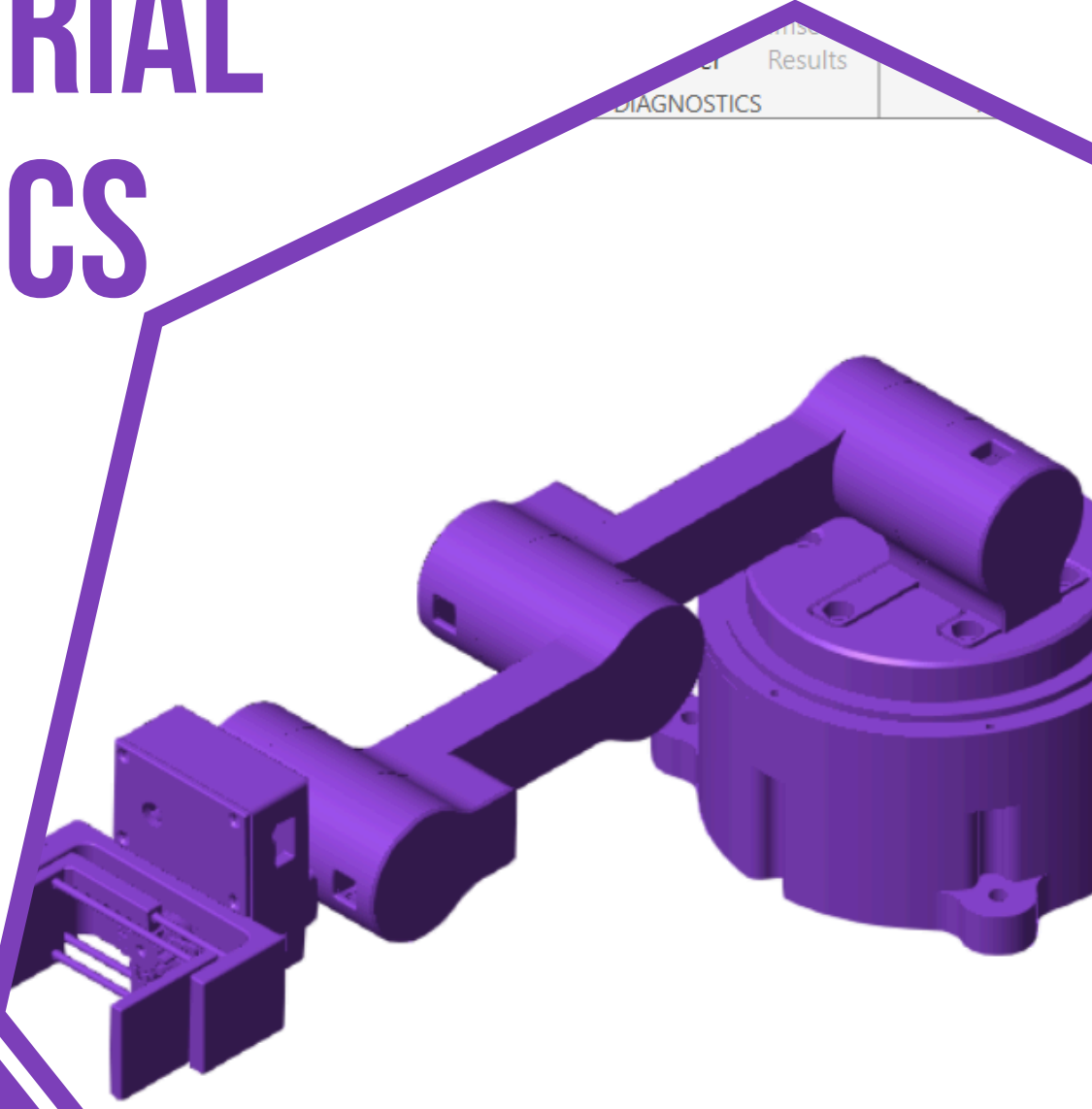
[MCT344]: INDUSTRIAL ROBOTICS

SPRING 2026



Project Submission: Analysis & Simulation of a robotic Manipulator

SECTION #01

TEAM #51

AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
Mechatronics Engineering Dept.

[MCT344]: Industrial Robotics

Project Submission

No.	Name	ID
1	Mohamed Hassan Mohamed Ali ElShall	2200130
2	Mokhtar Khalil Rafaat Ibrahim	2200552
3	Omar Wessam Farouk Ali	2200220
4	Philopateer Hany Nashaat Henawy	2200151

Table of Contents:

Table of Figures:

Table of Table:

1. Introduction:

Industrial robotic manipulators play an important role in modern manufacturing and automation. They provide high precision, repeatability, and efficiency, making them essential in many industrial applications. Studying these systems requires an understanding of kinematics, dynamics, trajectory planning, and control.

This report focuses on the **analysis and simulation** of an industrial robotic manipulator. Following the project requirements, we chose to study the robotic arm that our team previously designed and built in **[MCT 333]: Mechatronic Systems Design course**.

The robot "**Supernova**" is a **5-DOF serial robotic manipulator** featuring **4 revolute joints & End Effector**, designed for **pick-and-place operations** of boxes on pedestals. Where we are supposed to perform the following analysis on it as follows:

1. **Analytical Methods**: Where no predefined Matlab functions are used based on what was studied through our **[MCT344]: Industrial Robotics Course**
2. **Robotics Systems Toolbox:** Performing Analysis using Predefined functions provided by **Robotic Systems Toolbox** in Matlab
3. **Using Simscape Multibodies:** where the Manipulator's CAD is imported on Simscape Multibodies where Simulation and analysis is performed at the same time.

After that a comparison Table is carried out to compare between the result of the three methods on performing:

1. Workspace analysis **2. Forward Kinematics** **3. Inverse Kinematics**
4. Jacobian Analysis **5. Dynamic Analysis** **6. Trajectory Analysis**
7. Torque Control

To show how far the difference between the three methods to measure their accuracies, its complexity, etc.

2. Workspace Analysis:

2.1. Using Analytical Method:

To Get the Workspace Analysis using Analytical Solution, The following Matlab script was handled

```
% Supernova Robotic Arm Workspace – Analytical Method
% DH parameters: [a (mm), alpha (deg), d (mm), θ (deg)]
base_dz = 78;   % fixed base offset in Z
dh = [  0,  90, 45, 0;    % joint 1
      140,  0,  0, 0;     % joint 2
      130,  0,  0, 0;     % joint 3
      133,  0,  0, 0];    % joint 4

% Joint limits [min, max] in degrees
theta_lim = [-90 90;   % θ₁
              0 180;   % θ₂
             -90 90;   % θ₃
             -90 90];  % θ₄

N = 50000;   % number of random samples
% Generate random joint angles (radians)
lim1 = theta_lim(:,1).';
lim2 = theta_lim(:,2).';
theta = bsxfun(@plus,lim1,bsxfun(@times,(lim2-lim1),rand(N,4))) * pi/180;

% Forward kinematics – analytical, no toolboxes
P = zeros(N,3);
for i = 1:N
    T = eye(4);
    T = T * Tz(base_dz);
    T = T * DHtransform(dh(1,1), dh(1,2), dh(1,3), theta(i,1));
    T = T * DHtransform(dh(2,1), dh(2,2), dh(2,3), theta(i,2));
    T = T * DHtransform(dh(3,1), dh(3,2), dh(3,3), theta(i,3));
    T = T * DHtransform(dh(4,1), dh(4,2), dh(4,3), theta(i,4));
    P(i,:) = T(1:3,4)';
end

% Clipping the negative Z-values to make it more reasonable
P(P(:,3) < 0, 3) = 0;

% Plot workspace point cloud
scatter3(P(:,1), P(:,2), P(:,3));
axis equal; grid on;
xlabel('X (mm)'); ylabel('Y (mm)'); zlabel('Z (mm)');
title('4DOF Arm Workspace');

% --- Local functions ---
function T = DHtransform(a, alpha_deg, d, theta)
    alpha = alpha_deg * pi/180;
    T = [cos(theta), -sin(theta)*cos(alpha),  sin(theta)*sin(alpha), a*cos(theta);
         sin(theta),  cos(theta)*cos(alpha), -cos(theta)*sin(alpha), a*sin(theta);
              0,           sin(alpha),            cos(alpha),            d;
              0,               0,                     0,                 1];
end

function T = Tz(d)
    T = eye(4);
    T(3,4) = d;
end
```

Figure 1: Analytical Implementation to Perform Workspace Analysis

The Results was carried out as follows:

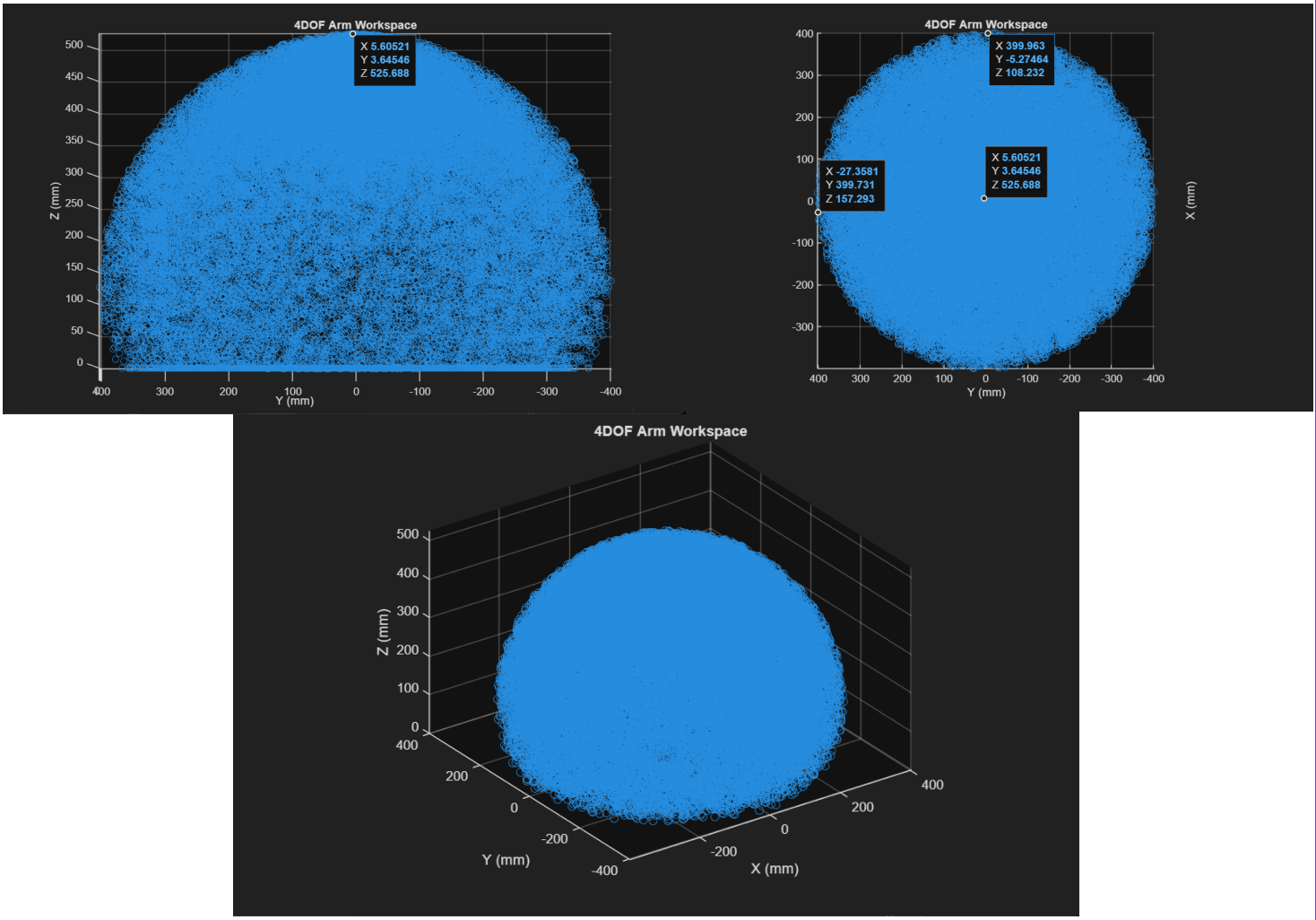


Figure 2: Results of the Workspace using Analytical Methods

2.2. Using Robotic Systems Toolbox:

Robotic Toolbox method is implemented using the following Script

```
clc;
%% 1. Load the Robot's Rigid Body Tree
SupernovaArm.DataFormat = 'row';
% Define the end-effector body name.
endEffectorName = 'Body5';

%% 2. Plot Robot's Workspace
rng default; % For reproducible results
fprintf('Generating reachable workspace\n');
[wksp, cfgs] = generateRobotWorkspace(SupernovaArm, {}, endEffectorName, ...
    'IgnoreSelfCollision', 'on');
fprintf('Generated %d reachable end-effector poses.\n', size(wksp, 1));
wksp(wksp(:,3) < 0, 3) = 0;
scatter3(wksp(:,1),wksp(:,2),wksp(:,3))
```

Figure 3: Script used to Generate The Robot's Workspace using Robotic Systems Toolbox

The Results Were as Follows:

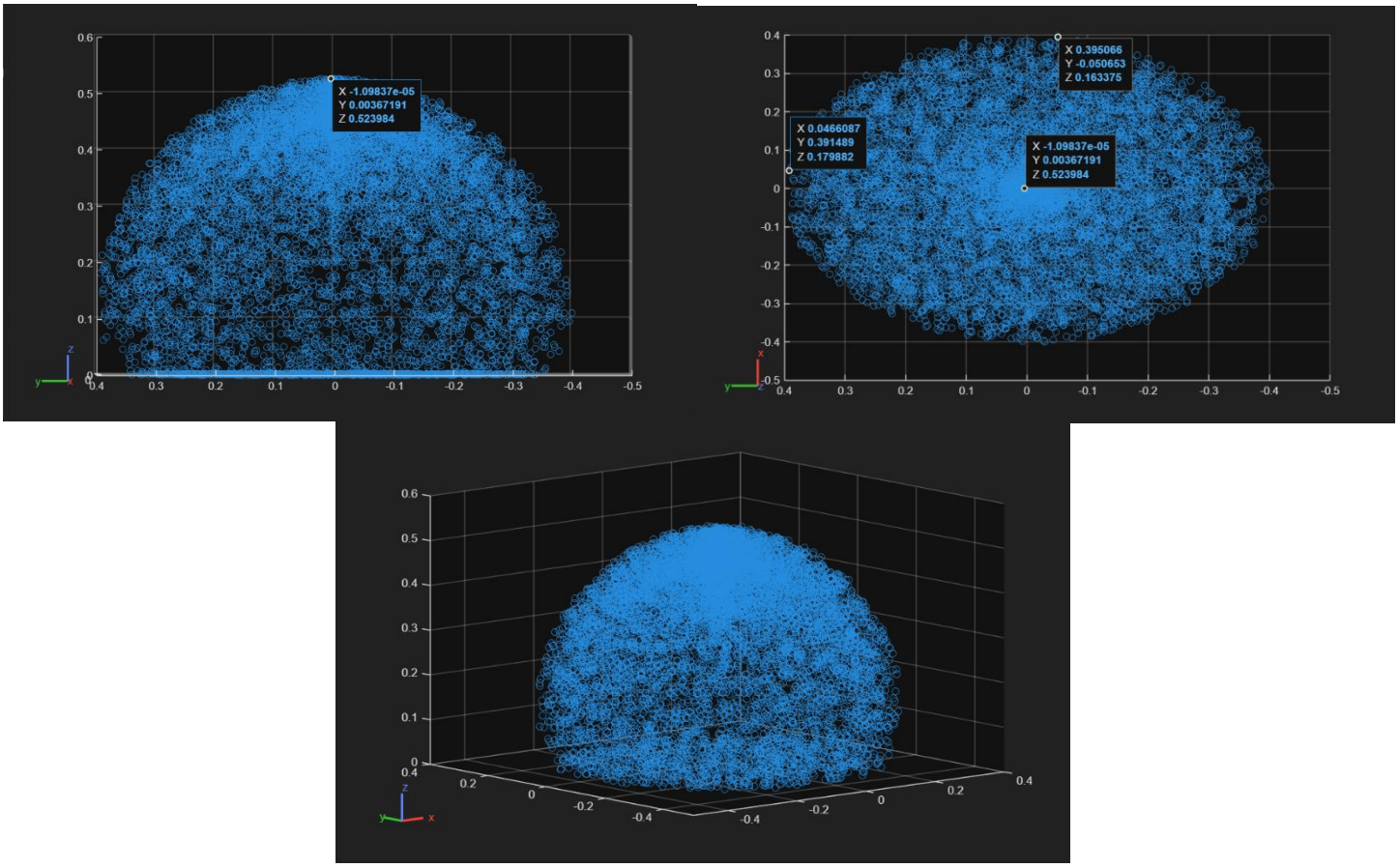


Figure 4: Results of the Workspace using Robotic Systems Toolbox

2.3. Using Simscape Multibodies®:

First the Joints' limits are assigned. Then a **Transform Sensor Block** is given to the end effector to monitor the end effector positions relative to the World Frame and exports them to the Workspace

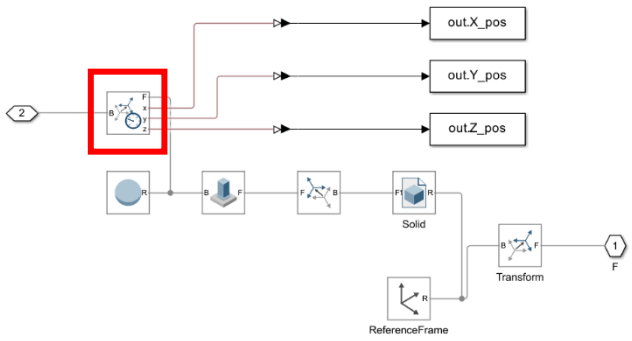


Figure 5: Assigning a Transform Sensor to the End Effector

Then, a **Uniform Random block** is given to each joint, where its **maximum and minimum values** is set to match the joint's limits, also the **time steps and seed numbers** are tuned to give better results. Finally the simulation is run at large stop time **(500 s)** to provide more points to enhance its accuracy

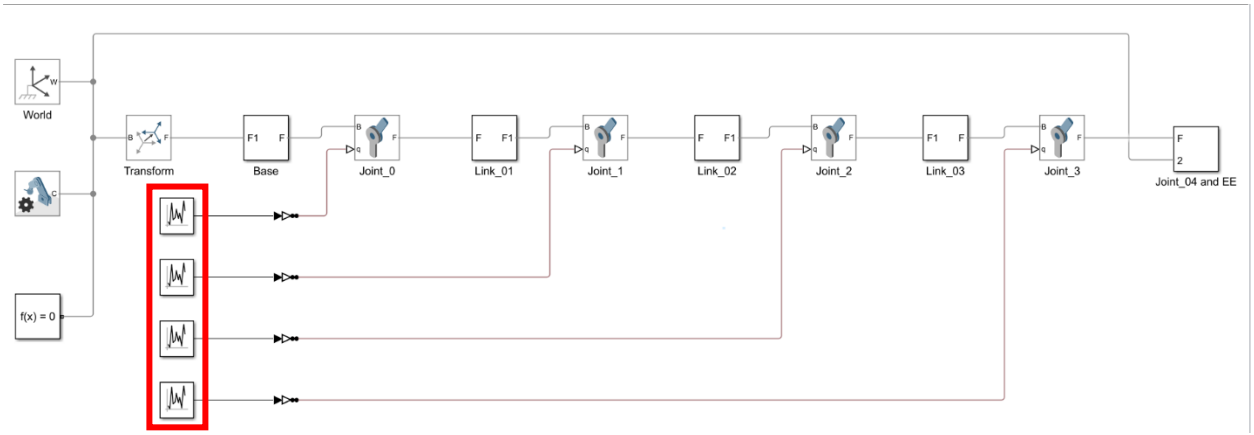


Figure 6: Giving the joint Random Configurations

After that, the (X,Y,Z) positions are plotted in a scatter 3D plot to give us the robot's workspace. Note that the negative Z-positions are clipped as it doesn't make any sense that the manipulator will go beyond the ground level

```
>> X_pos=out.X_pos;
>> Y_pos=out.Y_pos;
>> Z_pos=out.Z_pos;
>> Z_pos(Z_pos<0)=0;
>> scatter3(X_pos,Y_pos,Z_pos)
```

Figure 7: Plotting the Workspace Cloud

The workspace result were expressed as follows:

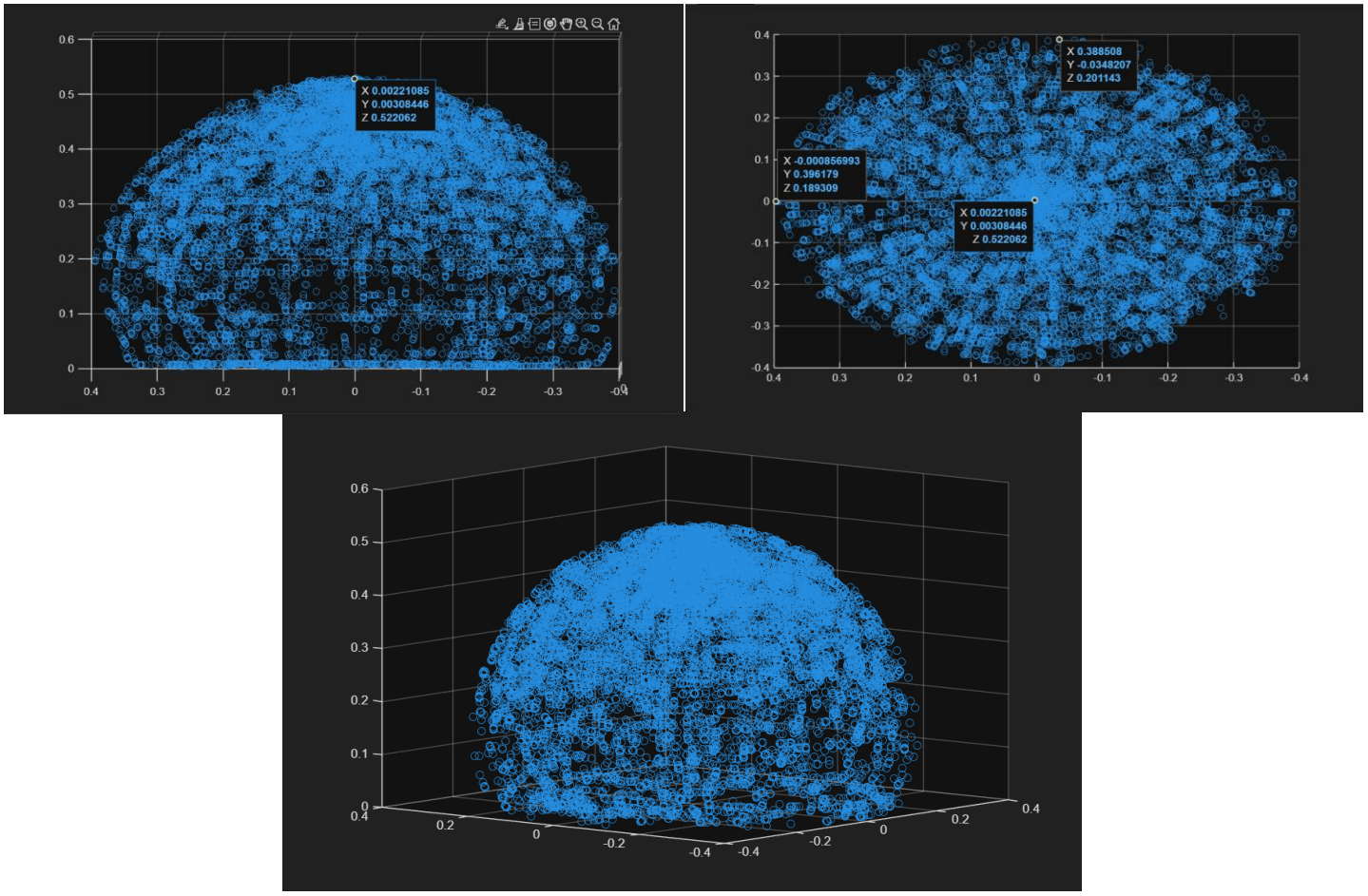


Figure 8: 3D scatter Plot showing the manipulator Workspace

2.4. Comparing and Validating between the Three methods:

From the CAD, we measured the distance between the center of the rotating base and the end of the end effector to get the maximum reachable distances across the three axes

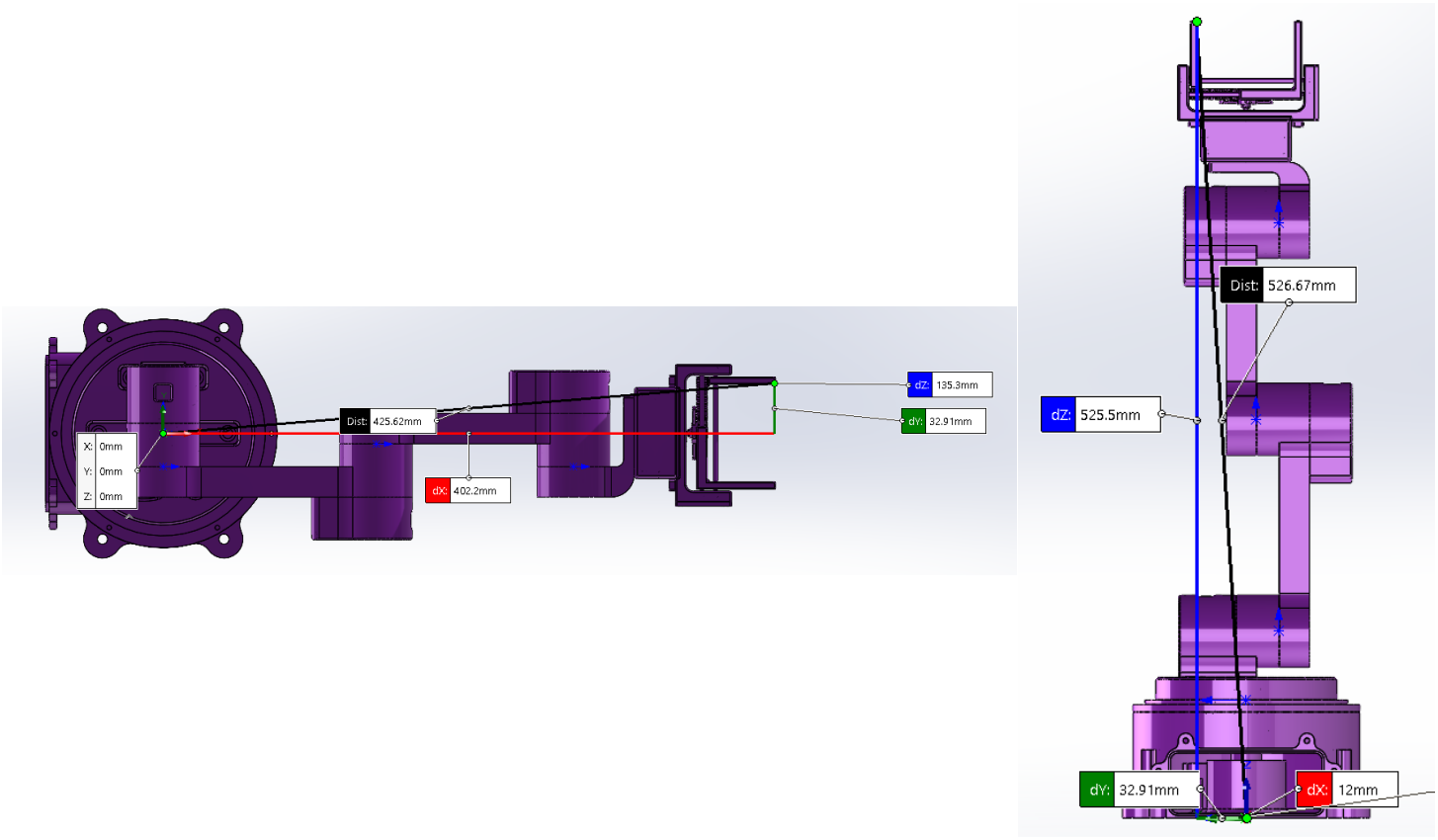


Figure 9: Measured Values from CAD

A following Comparison Table is carried out also to compare between them

Table 1: Comparison between the three methods based on Workspace Analysis

POC	Analytical Method	Robotics System Toolbox	Simulink/ Simscape Method®	CAD Values
Maximum X Value	399.963	395.066	388.508	402.200
Maximum Y Value	399.731	391.489	396.179	402.200
Maximum Z Value	525.688	523.984	522.062	525.500

From the following We can deduce that all of the procedures takes has given close results to the Actual data values from the CAD, hence the analysis is **valid**

3. Forward Kinematics Analysis:

3.1. Analytical Method:

The following Matlab script are carried out to estimate the Forward Kinematics based on the following DH-convention.

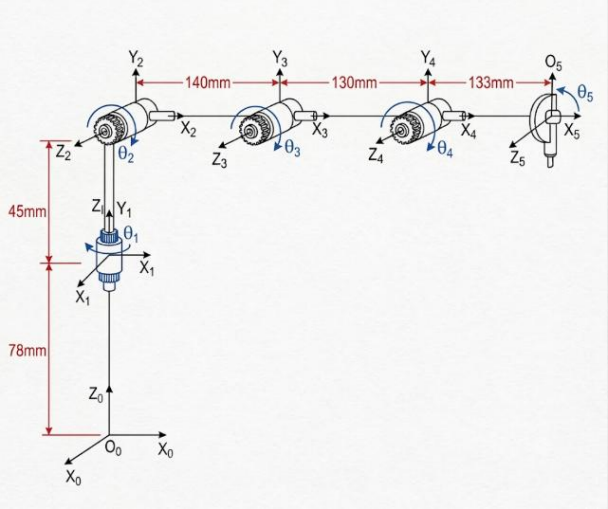


Figure 10: Coordinate Assignment for the Manipulator

Table 2: DH-Table for the Manipulator

Link	Joint	θ_z	d_z	a_x	α_x
0 → 0	-	0	78	0	0
0 → 1	1	θ_1	45	0	90^0
1 → 2	2	θ_2	0	140	0
2 → 3	3	θ_3	0	130	0
3 → 4	4	θ_4	0	133	0

```
% Forward KInematics, Analytical Method
%% 1. User Input
% Input Joint Angles [theta1, theta2, theta3, theta4]
theta = [0, pi/3, -pi/3, 0];

%% 2. Robot Parameters
base_dz = 78;   % fixed base offset in Z (mm)
% DH parameters: [a (mm), alpha (deg), d (mm)]
dh = [  0,  90, 45;    % joint 1
      140,   0,  0;    % joint 2
      130,   0,  0;    % joint 3
      133,   0,  0];   % joint 4

%% 3. Compute Forward Kinematics
T = eye(4);
T = T * Tz(base_dz);
T = T * DHtransform(dh(1,1), dh(1,2), dh(1,3), theta(1));
T = T * DHtransform(dh(2,1), dh(2,2), dh(2,3), theta(2));
T = T * DHtransform(dh(3,1), dh(3,2), dh(3,3), theta(3));
T = T * DHtransform(dh(4,1), dh(4,2), dh(4,3), theta(4));
% Extract and Display Results
P = T(1:3, 4)'; % End-effector position [X, Y, Z]
fprintf('Joint Angles (radians): [%.2f, %.2f, %.2f, %.2f]\n', theta_deg);
fprintf('End-Effector Position [X, Y, Z] (mm): [%.2f, %.2f, %.2f]\n\n', P);
disp('Final Transformation Matrix (T):');
disp(T);

%% 4. Helper Functions
% Function for Computing the A-Matrix
function T = DHtransform(a, alpha_deg, d, theta)
    alpha = alpha_deg * pi/180;
    T = [cos(theta), -sin(theta)*cos(alpha),  sin(theta)*sin(alpha), a*cos(theta);
         sin(theta),  cos(theta)*cos(alpha), -cos(theta)*sin(alpha), a*sin(theta);
              0,           sin(alpha),            cos(alpha),            d;
              0,              0,                     0,                  1];
end

% Function for Computing the Offset Transformation
function T = Tz(d)
    T = eye(4);
    T(3,4) = d;
end
```

Figure 12: Script Used to Get the Forward Kinematics

```
>> ForwardKinematics
Joint Angles (degrees): [0.00, 1.05, -1.05, 0.00]
End-Effector Position [X, Y, Z] (mm): [333.00, 0.00, 244.24]

Final Transformation Matrix (T):
    1.0000         0         0  333.0000
         0    0.0000   -1.0000    0.0000
         0    1.0000    0.0000  244.2436
         0         0         0    1.0000
```

Figure 11: The Result of the given script due to Forward Kinematic Analysis

3.2. Robotic Systems Toolbox® Method:

The following MatLab script is carried out to implement Forward kinematics using Robotic Systems Toolbox:

```
% Forward Kinematics using MATLAB Robotics System Toolbox
%% 1. Initializeing Robot's Configurations
SupernovaArm.DataFormat='row';
%Specify the joint angles
jointAngles = [0 -pi/6 -pi/3 0];
%Identify the end-effector body name
eeName = SupernovaArm.BodyNames{end};

%% 2. Compute Forward Kinematics
T = getTransform(SupernovaArm, jointAngles, eeName);
disp('--- Forward Kinematics Results ---');
fprintf('End-Effector Body Name: %s\n\n', eeName);
disp('4x4 Homogeneous Transformation Matrix (Base to End-Effector):');
disp(T);
% Extract and display just the Cartesian position (X, Y, Z) in meters
position = tform2trvec(T);
disp('Cartesian Position [X, Y, Z] (meters):');
disp(position);

%% 3. Visualize the Robot Configuration
figure;
show(SupernovaArm, jointAngles, 'PreservePlot', false);
title('Robot Configuration at Specified Joint Angles');
axis auto;
```

Figure 13: Script used to get Forward Kinematics using Robotic Systems Toolbox

And The Results were displayed as follows:

```
>> Forward_Kinematics
--- Forward Kinematics Results ---
End-Effector Body Name: Body5

4x4 Homogeneous Transformation Matrix (Base to End-Effector):
    1.0000   -0.0000   -0.0000    0.3319
   -0.0000    0.0000   -1.0000    0.0003
    0.0000    1.0000    0.0000    0.2385
         0         0         0    1.0000

Cartesian Position [X, Y, Z] (meters):
    0.3319    0.0003    0.2385
```

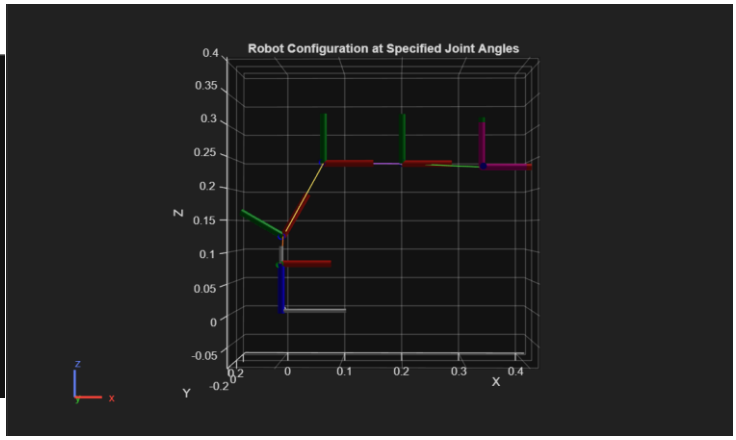


Figure 14: Shows the result of the forward Kinematics Using Robotic Systems Toolbox

3.3. Using Simscape/Multibodies®:

First, specific Joint Angles were Given to the Manipulator to calculate the Forward Kinematics, the given joint angles were adjusted to give the following position.

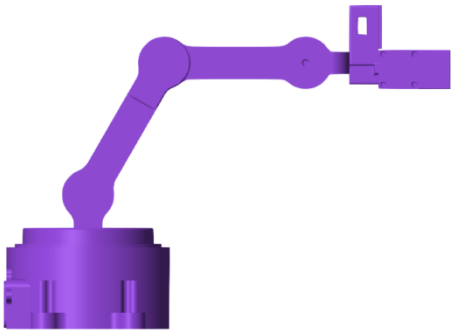


Figure 15: The Resulted position from the given joint angles

Then a **Get Transform Block** is introduced where the appropriate Rigid body tree is assigned, **the base and the target body** is set to get the relation between the base and the End effector. Then a **coordinate transformation convention** is introduced to transform between homogeneous transformation to extract the translational vectors only

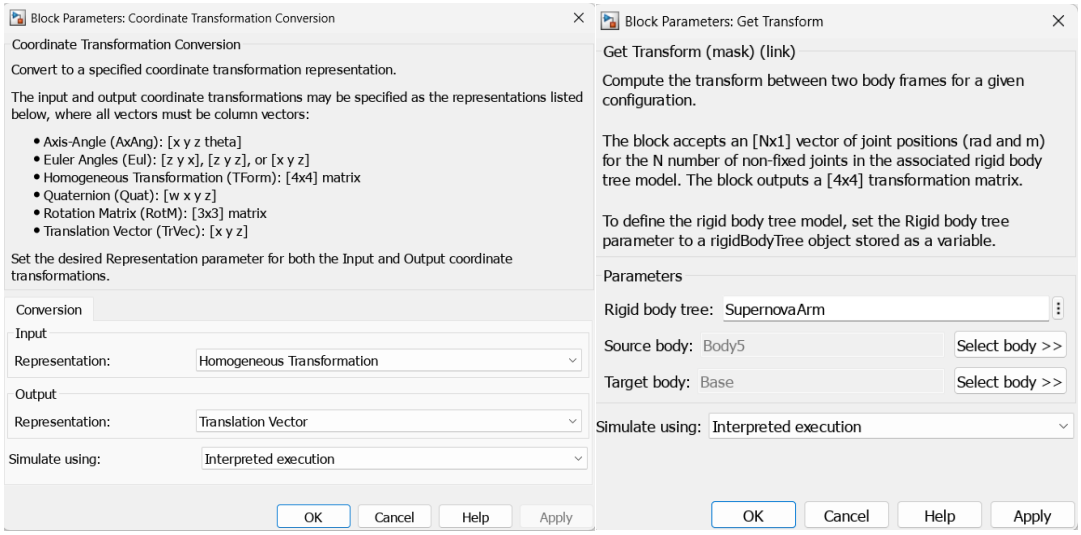


Figure 16: Adjusting Parameters for performing forward Kinematics

Using Display elements like Display and Scope blocks, Results of forward Kinematics are obtained

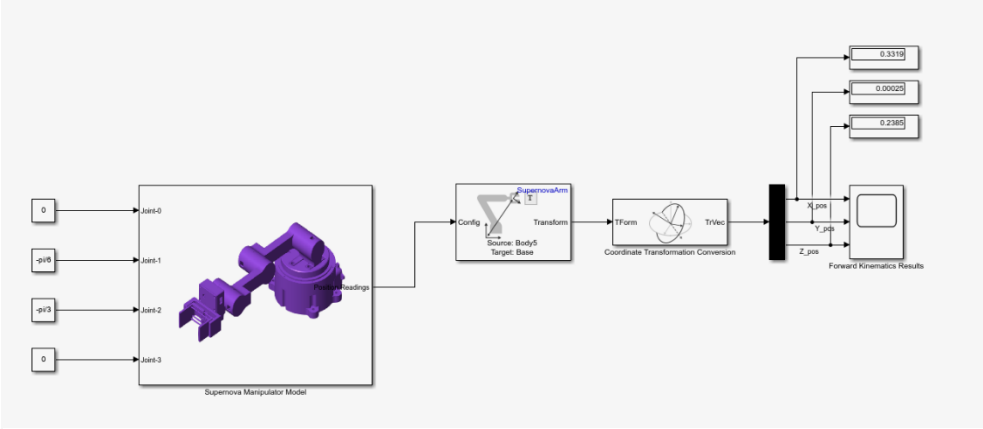


Figure 17: Forward Kinematics Implementation

The Result Were Plotted in the following scope

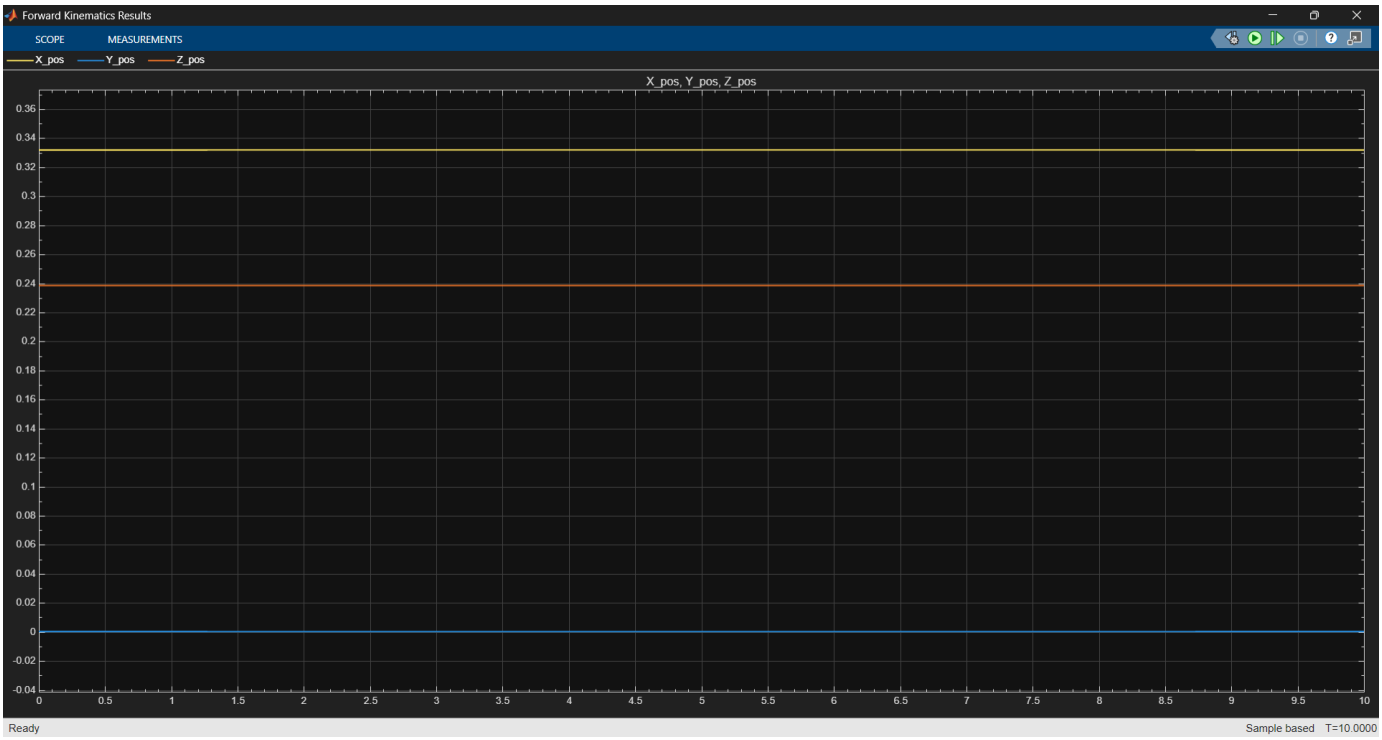


Figure 18: Scope Output of the Forward Kinematics

3.4. Comparing Among the three methods:

From the CAD, we measured the distance between the center of the rotating base and the end of the end effector to get the maximum reachable distances across the three axes for the same joint Configuration

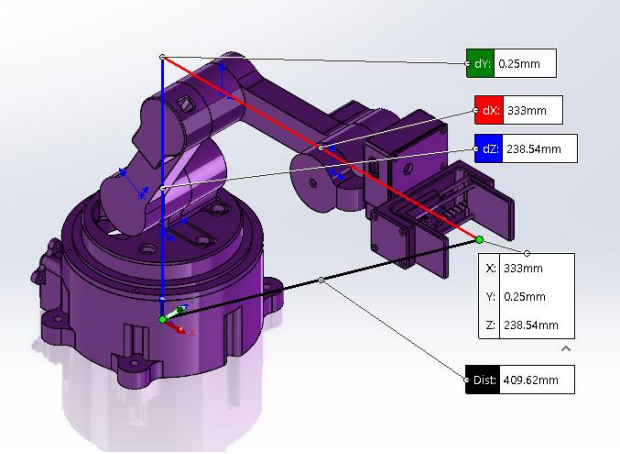


Figure 19: Measured Values from the CAD

The following Comparison table is carried out to compare between the three methods

Table 3: Comparison among the three methods to validate their Forward Kinematics

POC	Analytical Method	Robotics System Toolbox	Simulink/ Simscape Method®	CAD Values
X-Value	333.000	331.900	331.900	333.000
Y-Value	0.000	0.300	0.250	0.250
Z-Value	244.243	238.500	238.500	238.540

From the following We can deduce that all of the procedures takes has given close results to the Actual data values from the CAD, hence the analysis is **valid**

4. Inverse Kinematics

4.1. Analytical Method:

A Geometrical Approach is carried out to solve the inverse kinematics problem for the Manipulator. Taking a test case where Manipulator is given the required position and orientation to solve the Kinematic Problem

$$X_{req} = 200\ mm$$
$$Y_{req} = 200\ mm$$
$$Z_{req} = 300\ mm$$
$$\theta_{Pitch} = 0^0$$

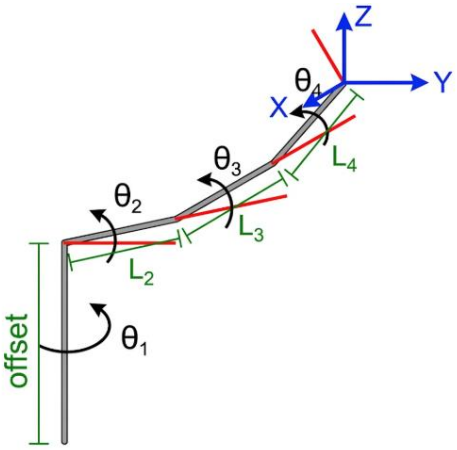


Figure 20: Geometrical Approach to solve the Inverse Kinematics of the Arm

The following MATLAB Script is carried out to solve the Inverse Kinematics Solution, and the result was as follows:

```
%% ========================Inverse Kinematics============================
%  All Dimensions are in mm
%% 1. Setting Parameters and Requirements
% Links Length
  Offset=123; L2=140;  L3=130;  L4=133;
% Given Requirements: P.S. Pitch = Theta2+ Theta3 + Theta 4
  X_pos=200;  Y_pos=200;  Z_pos=300;  Pitch=0;
%% 2. Getting Joint Variables
Theta_1= atan2d(Y_pos,X_pos);                % Rotating Base Angle
R=sqrt(X_pos^2+Y_pos^2);                     % Radial Distane
Z_arm= Z_pos-Offset;
%% 3. Getting Arm Wrist
R_w=R-L4*cos(Pitch);
Z_w=Z_arm-L4*sin(Pitch);
D_w=sqrt(R_w^2+Z_w^2);
Theta_3= acosd((D_w^2-L2^2-L3^2)/(2*L2*L3));   % Wrist Angle
%% 4. Getting Shoulder Angle
Theta_2=atan2d(Z_w,R_w)-atan2d(L3*sind(Theta_3),L2+L3*cosd(Theta_3));
%% 5. Getting The last Joint Angle
Theta_4=Pitch-Theta_3-Theta_2;
%% 6. Display Results
fprintf('Inverse Kinematics Solutions\n');
fprintf('[%.2f, %.2f, %.2f, %.2f]\n', Theta_1,Theta_2,Theta_3,Theta_4);
```

```
>> InverseKinematics
Inverse Kinematics Solutions
[45.00, 20.19, 61.66, -81.84]
```

Figure 21: Matlab Script used to solve the Inverse Kinematics of the manipulator

Verifying the resulted Joint Angles using the Forward Kinematics Script, the Results were as follows
The position of the End Effector is almost the same as the required position given above. Hence, the given angles are **correct**

```
>> InverseKinematics
Inverse Kinematics Solutions
[45.00, 20.19, 61.66, -81.84]
>> ForwardKinematics
Joint Angles (radians): [0.79, 0.35, 1.08, -1.43]
End-Effector Position [X, Y, Z] (mm): [199.99, 199.99, 300.03]

Final Transformation Matrix (T):
    0.7071   -0.0001    0.7071  199.9888
    0.7071   -0.0001   -0.7071  199.9888
    0.0002    1.0000    0.0000  300.0291
         0         0         0    1.0000
```

Figure 22: Verifying Inverse Kinematics using Forward Kinematics

4.2. Using Robotic Systems Toolbox:

The same positions and orientation requirements are given to the manipulator. Also an assigned **weights for the positions and orientations** are given to the Manipulator to match the Analytical method's requirements. Then the Following Matlab Script is carried out as follows:

```
%% ==========Inverse Kinematics Using Robotic Systems Toolbox=============
%% 1. Initialize the Inverse Kinematics Solver
ik = inverseKinematics('RigidBodyTree', SupernovaArm);
targetPosition = [0.2, 0.2, 0.3];
targetOrientation = eul2quat([0, 0, 0]);
targetPose = trvec2tform(targetPosition) * quat2tform(targetOrientation);
%% 2. Define Solver Weights
weights = [0, 1, 0, 1, 1, 1];
%Set the Initial Guess
initialGuess = homeConfiguration(SupernovaArm);
%% 3. Execute the IK Solver
disp('Calculating Inverse Kinematics for Supernova Arm...');
[configSoln, solnInfo] = ik(endEffectorName, targetPose, weights, initialGuess);
%% 4. Output the Results
disp('--- Solver Output ---');
disp(['Status: ', solnInfo.Status]);
disp(['Iterations: ', num2str(solnInfo.Iterations)]);
disp(['Pose Error: ', num2str(solnInfo.PoseErrorNorm)]);
disp('--- Joint Angles (degrees) ---');

for i = 1:length(configSoln)
    fprintf('Joint %d: %.2f°\n', i, rad2deg(configSoln(i)));
end
%% 5.Visualization
figure;
show(SupernovaArm, configSoln);
title('Supernova Arm: IK Solution');
axis equal;
hold on;
plotTransforms(targetPosition, targetOrientation, 'FrameSize', 0.15);
hold off;
```

```
>> Inverse_Kinematics
Calculating Inverse Kinematics for Supernova Arm...
--- Solver Output ---
Status: success
Iterations: 310
Pose Error: 3.7796e-09
--- Joint Angles (degrees) ---
Joint 1: -44.95°
Joint 2: -25.83°
Joint 3: -81.28°
Joint 4: 62.06°
```

Figure 23: Matlab script carried out to solve the Inverse Kinematics

The results are displayed and Visualized as follows:

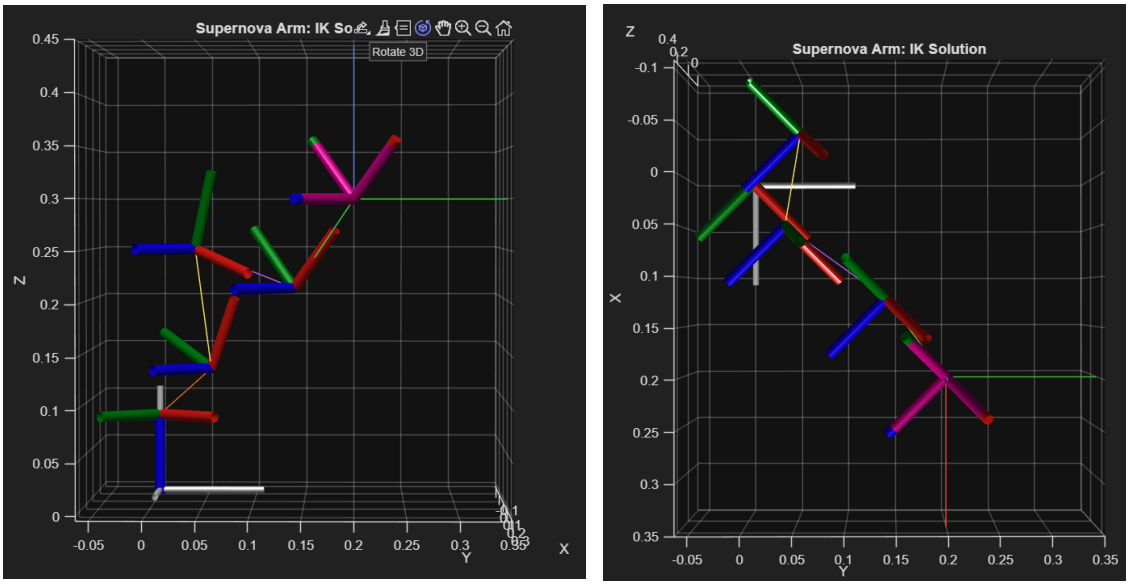


Figure 24: Visualization of the Manipulator's Orientation using the Joint Angles resulted from Inverse Kinematics Solutions

4.3. Simscape Multibodies® Method:

An Inverse Kinematics Block is Added to the Model to compute the necessary Joint Angles. Weights and Initial Guesses are made based on the Joint Home configuration. The requirement positions are placed the same as the other two method for the sake of comparison.

The Joint Angles' values were carried out from the Model Subsystem:

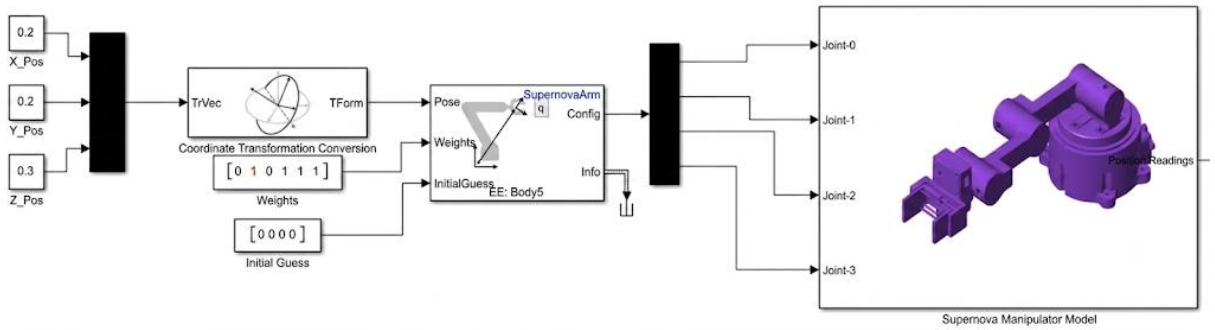


Figure 25: Inverse Kinematic Analysis using Simscape

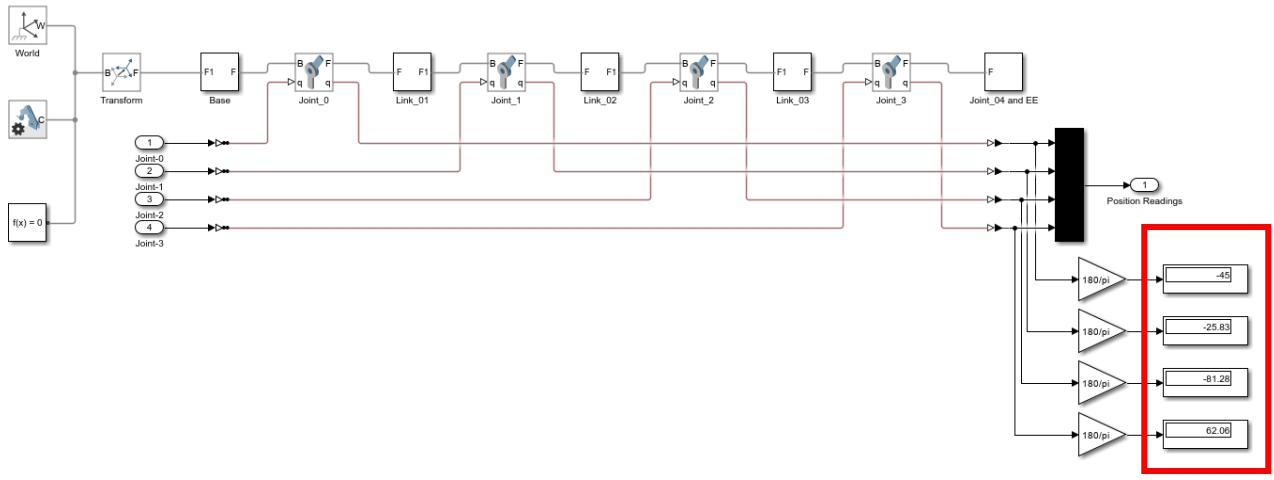


Figure 26: Getting the Joint Angles after solving Inverse Kinematics

Also, to verify the correct position of the Manipulator, the X,Y,Z positions are carried out from the **Forward Kinematics Subsystem**

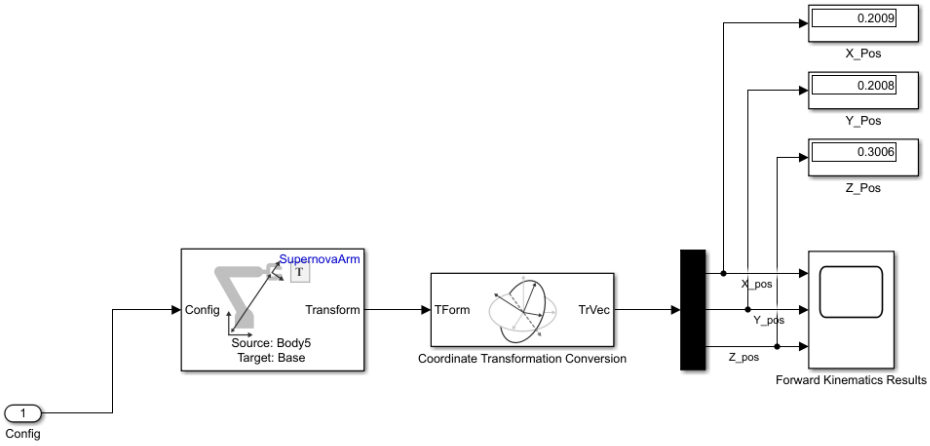


Figure 27: Verifying the Joint Angles using Forward Kinematics

Finally, the Robot is Visualized as follows:

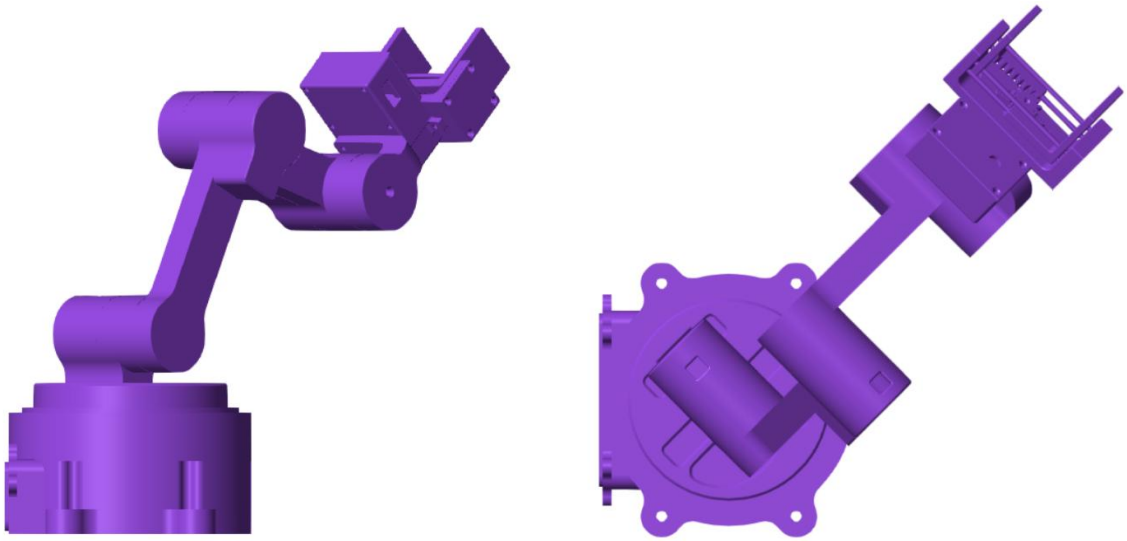


Figure 28: Visualization of the Manipulator after Solving Inverse Kinematics

4.4. Comparison Between the Three Methods:

The following comparison are carried out to compare between the results of the three methods used to solve Inverse Kinematics.

Table 4: Comparison between Inverse Kinematics Results based onto the three method

POC	Analytical Method	Robotics System Toolbox	Simulink/ Simscape Method®
θ_1	45.00^0	-44.95^0	-45^0
θ_2	20.19^0	-25.83^0	-25.83^0
θ_3	61.66^0	-81.28^0	-81.28^0
θ_4	-81.84^0	62.06^0	-62.06^0

As we observe, there is a bit of a difference between the results for the angles using the analytical method and both by using Robotic Systems Toolbox and Simscape Multibodies Method. And that is due to many reasons:

1. A Manipulator can have **more than one Inverse Kinematic solution** for the same position and orientation, where it eliminates its singularity and improves the manipulator's dexterousity
2. **In both Robotics Toolbox method and Simscape Multibodies methods**: there may be an inversion in angle's directions due to Multibodies configurations in MATLAB, where the value of the angle may be correct in magnitude but have opposite direction "like θ_1"
3. **In both Robotics Toolbox method and Simscape Multibodies methods**: the method of solving is numerically based on the weights and the given initial guess. While during analytical method, its solution is geometrical.

5. Jacobian Matrix:

5.1. Analytical Solution:

The following MATLAB Script is carried out to get the Jacobian Matrix for the Manipulator.

```
%% 1. Performing Forward Kinematics
theta = [0, pi/3, -pi/3, 0];
% DH parameters: [a, alpha, d]
base_dz = 78;
dh = [  0,  90, 45;    % Link 1
      140,   0,  0;    % Link 2
      130,   0,  0;    % Link 3
      133,   0,  0];   % Link 4
% Forward Kinematics Transformation Matrix
T01 = Tz(base_dz) * DHtransform(dh(1,1), dh(1,2), dh(1,3), theta(1));
T02 = T01 * DHtransform(dh(2,1), dh(2,2), dh(2,3), theta(2));
T03 = T02 * DHtransform(dh(3,1), dh(3,2), dh(3,3), theta(3));
T04 = T03 * DHtransform(dh(4,1), dh(4,2), dh(4,3), theta(4));
% Extract Positions (o_i)
o0 = [0; 0; 0];
o1 = T01(1:3, 4);
o2 = T02(1:3, 4);
o3 = T03(1:3, 4);
on = T04(1:3, 4); % End-effector tip
% Extract Orientations about Z-Axis (z_i)
z0 = [0; 0; 1];
z1 = T01(1:3, 3);
z2 = T02(1:3, 3);
z3 = T03(1:3, 3);
%% 2. Getting Jacobian Matrix
J1 = [cross(z0, (on - o0)); z0];
J2 = [cross(z1, (on - o1)); z1];
J3 = [cross(z2, (on - o2)); z2];
J4 = [cross(z3, (on - o3)); z3];
J = [J1, J2, J3, J4];
disp('--- JACOBIAN MATRIX ---');
disp(J);
```

```
>> JacobianMatrix
--- JACOBIAN MATRIX ---
   -0.0000 -121.2436         0         0
  333.0000    0.0000    0.0000    0.0000
         0  333.0000  263.0000  133.0000
         0         0         0         0
         0   -1.0000   -1.0000   -1.0000
    1.0000    0.0000    0.0000    0.0000
```

Figure 29: Getting Jacobian Matrix Using Analytical Method

After that a singularity analysis is carried out to check if this position is singular or not

```
%% 3. Singularity Analysis:
disp('---- 2. Singularity Analysis ----')
J11=J(1:2,1:2);         det_J11=det(J11);
J12=J(1:2,3:4);         det_J12=det(J12);
J21=J(3:4,1:2);         det_J21=det(J21);
J22=J(3:4,3:4);         det_J22=det(J22);
J31=J(5:6,1:2);         det_J31=det(J31);
J32=J(5:6,3:4);         det_J32=det(J32);
disp('Determinant of the Jacobian Matrix:')
Det_matrix=[det_J11, det_J12; det_J21, det_J22; det_J31, det_J32]
if(Det_matrix==zeros(3,2))
    disp('Singular Configuration')
else
    disp('Non-Singular Configuration')
end
```

```
---- 2. Singularity Analysis ----
Determinant of the Jacobian Matrix:

Det_matrix =

   1.0e+04 *

    4.0374         0
         0         0
    0.0001         0

Non-Singular Configuration
```

Figure 30: Singularity Analysis for the Manipulator using Analytical Method

Finally, A velocity Mapping is performed to link between the velocities in the Task Space and the velocities in the Joint Space

```
48  %% 4. Velocity Mapping
49  q_dot = [0.1; 0.2; -0.1; 0.1]; % Example joint speeds (rad/s)
50  x_dot = J * q_dot;             % Forward mapping: q_dot -> x_dot
51  J_pinv = pinv(J);              % Inverse Mapping: x_dot -> q_dot
52  q_dot_recovered = J_pinv * x_dot;
53  disp('--- 3. VELOCITY MAPPING TEST ---');
54  disp('Assumed Joint Velocities (q_dot):');
55  disp(q_dot);
56  disp('Resulting End-Effector Task Velocities (x_dot):');
57  disp(x_dot);
58  disp('Recovered Joint Velocities (J_pinv * x_dot):');
59  disp(q_dot_recovered);
60
```

```
--- 3. VELOCITY MAPPING TEST ---
Assumed Joint Velocities (q_dot):
    0.1000
    0.2000
   -0.1000
    0.1000

Resulting End-Effector Task Velocities (x_dot):
  -24.2487
   33.3000
   53.6000
         0
   -0.2000
    0.1000

Recovered Joint Velocities (J_pinv * x_dot):
    0.1000
    0.2000
   -0.1000
    0.1000
```

Figure 31: Velocity Mapping using Analytical Methods

5.2. Using Robotic Systems Toolbox:

The following Matlab Script Illustrates the Obtaining of the Jacobian Matrix

```
%% 1. Rigid Body Tree Setup
SupernovaArm.DataFormat = 'column';
q_given = [0; -pi/6; -pi/3; 0];
q_dot_given = [0.1; 0.2; -0.1; 0.1];
config = q_given;
%% 2. Jacobian Matrix Determination
J = geometricJacobian(SupernovaArm, config, eeName);
% This Function gets the angular component before the linear component
% so we adjust them
J_v=J(4:6,:);
J_w=J(1:3,:);
J_mod=[J_v; J_w];
disp('--- 1. Jacobian Analysis ---');
disp('Jacobian matrix J (6 x N):');
disp(J_mod);
```

```
--- 1. Jacobian Analysis ---
Jacobian matrix J (6 x N):
    0.0003   -0.1152    0.0060    0.0060
   -0.3319    0.0000   -0.0000   -0.0000
    0.0000    0.3319    0.2619    0.1319
         0   -0.0000   -0.0000   -0.0000
   -0.0000   -1.0000   -1.0000   -1.0000
   -1.0000    0.0000    0.0000    0.0000
```

Figure 32: Getting the Jacobian Analysis using Robotic Systems Toolbox

Then Singularity Analysis is Performed onto the Jacobian Matrix where it **computes the rank of the matrix** if it is **below the maximum possible rank**, a singularity is detected

```
%% 3. Singularity Analysis
n = size(J, 2);          % number of joints (DOF)
maxRank = min(6, n);     % maximum possible rank
% Rank Test
rank_J = rank(J);
fprintf('Jacobian Rank is: %d\n', rank_J);
fprintf('Maximum Rank is: %d\n', maxRank);
if rank_J < maxRank
    fprintf('*** SINGULAR CONFIGURATION DETECTED ***\n');
    fprintf('Jacobian rank = %d (max possible = %d)\n', rank_J, maxRank);
else
    fprintf('Configuration is non-singular. Jacobian rank = %d\n', rank_J);
end
```

```
Jacobian Rank is: 4
Maximum Rank is: 4
Configuration is non-singular. Jacobian rank = 4
```

Figure 33: Singularity Analysis for the Manipulator using Robotic Systems Toolbox

Finally, a velocity Mapping is performed to link the task space velocities with the joint space velocities

```
%% 4. Velocity Mapping
% Map joint velocities to end-effector velocities using V = J * q_dot
V_ee = J * q_dot_given;
% Separate angular and linear velocities for clarity
omega = V_ee(1:3); % Angular velocity [rad/s]
v= V_ee(4:6); % Linear velocity [m/s]
disp('--- 3. Velocity Mapping ---');
disp('Given Joint Velocities (q_dot):');
disp(q_dot_given);
disp('End-Effector Velocity (V = J * q_dot):');
fprintf('Angular Velocity [wx, wy, wz]^T : [%.4f, %.4f, %.4f] rad/s\n' ...
    , omega(1), omega(2), omega(3));
fprintf('Linear Velocity  [vx, vy, vz]^T : [%.4f, %.4f, %.4f] m/s\n' ...
    , v(1), v(2), v(3));
```

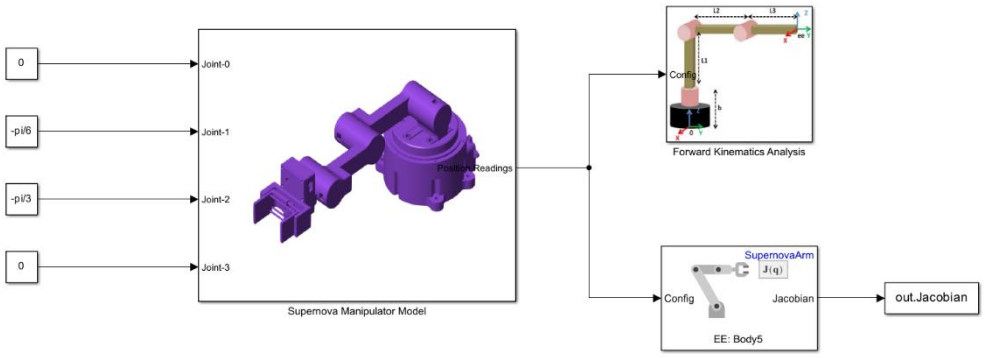
```
--- 3. Velocity Mapping ---
Given Joint Velocities (q_dot):
    0.1000
    0.2000
   -0.1000
    0.1000

End-Effector Velocity (V = J * q_dot):
Angular Velocity [wx, wy, wz]^T : [-0.0000, -0.2000, -0.1000] rad/s
Linear Velocity  [vx, vy, vz]^T : [-0.0230, -0.0332, 0.0534] m/s
>>
```

Figure 34: Velocity Mapping using Robotic Systems Toolbox

5.3. Using Simscape Multibodies Method:

We used the same angle input to get the Jacobian Matrix for the Manipulator




```
>> Jacobian

Jacobian =

         0   -0.0000   -0.0000   -0.0000
   -0.0000   -1.0000   -1.0000   -1.0000
   -1.0000    0.0000    0.0000    0.0000
    0.0003   -0.1152    0.0060    0.0060
   -0.3319    0.0000   -0.0000   -0.0000
    0.0000    0.3319    0.2619    0.1319

>> J_v=Jacobian(4:6,:);
>> J_w=Jacobian(1:3,:);
>> Jacobian=[J_v;J_w]

Jacobian =

    0.0003   -0.1152    0.0060    0.0060
   -0.3319    0.0000   -0.0000   -0.0000
    0.0000    0.3319    0.2619    0.1319
         0   -0.0000   -0.0000   -0.0000
   -0.0000   -1.0000   -1.0000   -1.0000
   -1.0000    0.0000    0.0000    0.0000
```

Figure 35: Getting the Jacobian Matrix using Simscape Multibodies Method

Singularity Analysis is performed where it detects if the **Jacobian Matrix Rank** is less than the maximum Rank "4".

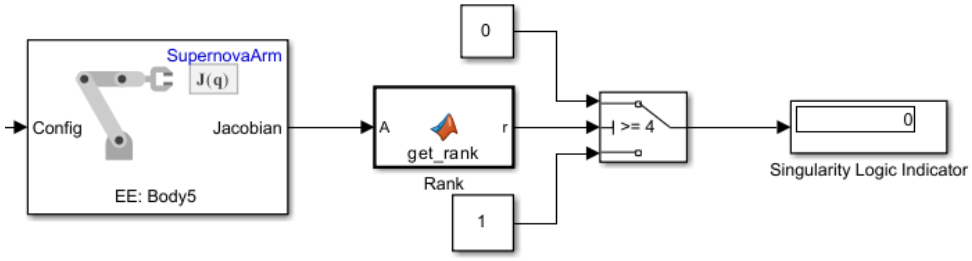


Figure 36: Singularity Analysis Checking using Simulink

Finally, A velocity Mapping is carried out for the same Joint Velocities to get the Task space Velocities

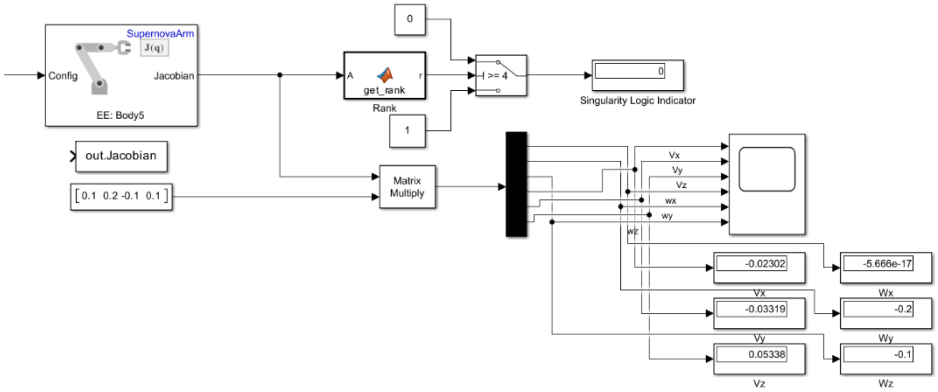


Figure 37: Velocity Mapping using Simscape Multibodies

Where the Final Values of the Task Space Linear and Angular Velocities are expressed as follows

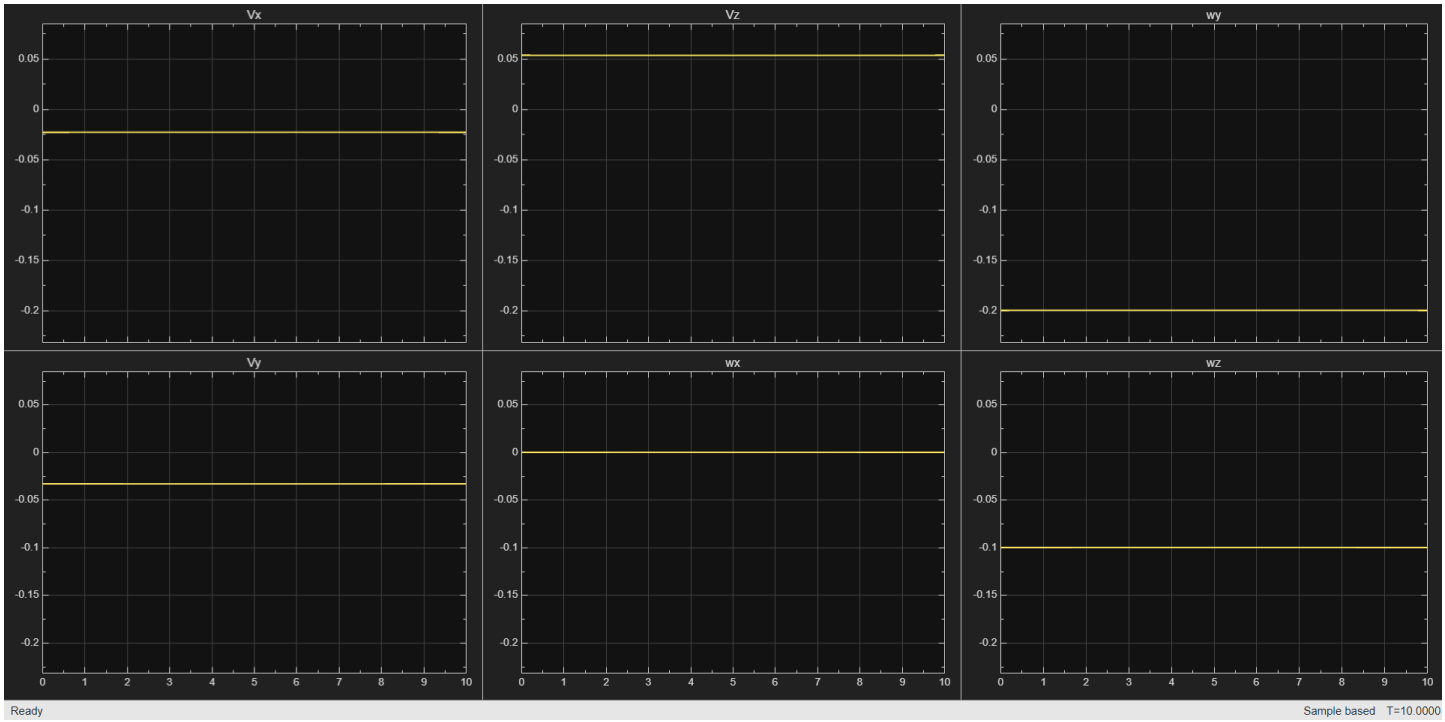


Figure 38: Result of the Velocity Mapping of the Task Space

5.4. Comparison Between the Three Methods:

The following Comparison table is carried out to perform Jacobian Analysis among the three methods

Table 5: Comparison between the Jacobian Analysis Results among the three methods

POC	Analytical Method	Robotics System Toolbox	Simulink/ Simscape Method®
Jacobian Matrix	>> JacobianMatrix --- JACOBIAN MATRIX --- -0.0000 -121.2436 0 0 333.0000 0.0000 0.0000 0.0000 0 333.0000 263.0000 133.0000 0 0 0 0 0 -1.0000 -1.0000 -1.0000 1.0000 0.0000 0.0000 0.0000	--- 1. Jacobian Analysis --- Jacobian matrix J (6 x N): 0.0003 -0.1152 0.0060 0.0060 -0.3319 0.0000 -0.0000 -0.0000 0.0000 0.3319 0.2619 0.1319 0 -0.0000 -0.0000 -0.0000 -0.0000 -1.0000 -1.0000 -1.0000 -1.0000 0.0000 0.0000 0.0000	Jacobian = 0.0003 -0.1152 0.0060 0.0060 -0.3319 0.0000 -0.0000 -0.0000 0.0000 0.3319 0.2619 0.1319 0 -0.0000 -0.0000 -0.0000 -0.0000 -1.0000 -1.0000 -1.0000 -1.0000 0.0000 0.0000 0.0000
Singularity	Non-Singular Configuration		
V_X	-0.0242	-0.0230	-0.02302
V_Y	0.0333	-0.0332	-0.03319
V_Z	0.0536	0.0534	0.05338
ω_x	0.0000	0.0000	0.00000
ω_y	-0.2000	-0.2000	-0.20000
ω_z	0.1000	0.1000	-0.10000

From The following table, we deduce that the values are close to each other, except for some sign changes due to Rigid body tree configuration in MATLAB. Hence, the calculation is **valid**.

6. Dynamic Analysis:

6.1. Analytical Method:

The following is carried out to perform the dynamic Analysis onto the manipulator.

First, **Symbols and DH parameters** are initialized in order to get the required variables

```
%% Dynaminc Analysis Analytical Method
%% 1. Given Parameters
thetas    = [0, pi/3, -pi/3, 0];
theta_dot = [0.1, 0.2, -0.1, -0.1];
masses    = [0.17, 0.15623, 0.15443, 0.19014];
I1 = [222364.10,   503.01,   4909.49;
        503.01, 237697.08,  21236.13;
       4909.49,  21236.13, 213106.79] * 1e-9;
I2 = [408579.31,  17507.74,      0.00;
       17507.74,  59808.96,     -0.05;
           0.00,     -0.05, 434533.61] * 1e-9;
I3 = [354064.76,  15434.47,      0.00;
       15434.47,  59679.89,     -0.04;
           0.00,     -0.04, 380063.53] * 1e-9;
I4 = [331327.56, -72734.27,  -2635.64;
      -72734.27, 193326.46, -18877.39;
       -2635.64, -18877.39, 401879.53] * 1e-9;
I_tensors = {I1, I2, I3, I4};
%% 2. Symbolic Variables & DH Parameters
syms q1 q2 q3 q4 dq1 dq2 dq3 dq4 real
q = [q1; q2; q3; q4];
dq = [dq1; dq2; dq3; dq4];
% Fixed base transformation (Link 0 -> 0)
T_base = [eye(3), [0; 0; 78]; 0 0 0 1];
% Standard DH Table: [alpha_x, a_x, d_z, theta_z]
DH = [pi/2,   0, 45, q1;
         0, 140,  0, q2;
         0, 130,  0, q3;
         0, 133,  0, q4];
```

Figure 39: Initializing the symbols and the DH Parameters

Second, **Forward Kinematics** are carried out to get the required matrices for estimating the Jacobian Matrix

```
%% 3. Forward Kinematics
T = cell(1, 4);  % Transformation Matrix
R = cell(1, 4);  % Rotation Matrix
P = cell(1, 4);  % Position Matrix
Z = cell(1, 4);  % Z-axis Matrix
T_curr = T_base;
for i = 1:4
    alpha = DH(i, 1); a = DH(i, 2); d = DH(i, 3); theta = DH(i, 4);
    % Denavit-Hartenberg Transformation Matrix
    A = [cos(theta), -sin(theta)*cos(alpha),  sin(theta)*sin(alpha), a*cos(theta);
         sin(theta),  cos(theta)*cos(alpha), -cos(theta)*sin(alpha), a*sin(theta);
                  0,             sin(alpha),             cos(alpha),           d;
                  0,                      0,                      0,           1];
    T_curr = T_curr * A;
    T{i} = T_curr;
    % Extract Rotation, Position, and Z-axis
    R{i} = T_curr(1:3, 1:3);
    P{i} = T_curr(1:3, 4);
    if i == 1
        Z{1} = T_base(1:3, 3);
    else
        Z{i} = T{i-1}(1:3, 3);
    end
end
```

Figure 40: Forward Kinematics Implementation

Third, **Jacobian Matrix** is Calculated to get the **Inertia Matrix** as follows were,

$$M(q) = \sum_{i=1}^{4} J_{v_i}^T m_i J_{v_i} + J_{\omega_i}^T I_i J_{\omega_i}$$

```
%% 4. Getting Jacobian Matricies and Getting the Inertia Matrix M(q):
disp('Calculating the Inertia Matrix...');
M_sym = sym(zeros(4, 4));
for i = 1:4
    J_v = sym(zeros(3, 4));
    J_w = sym(zeros(3, 4));
    % Calculate Jacobians up to link i
    for j = 1:i
        if j == 1
            P_prev = T_base(1:3, 4);
        else
            P_prev = P{j-1};
        end
        J_v(:, j) = cross(Z{j}, P{i} - P_prev); % Linear Velocity Jacobian
        J_w(:, j) = Z{j};                       % Angular Velocity Jacobian
    end
    % Inertia Tensor in World Frame = R * I * R^T
    I_world = R{i} * I_tensors{i} * R{i}.';
    % Accumulate Mass Matrix components
    M_sym = M_sym + masses(i) * (J_v.' * J_v) + (J_w.' * I_world * J_w);
end
M_sym = simplify(M_sym);
```

Figure 41: Getting Jacobian Matrix and the Inertial Matrix

Fourth, a **Gravity vector** is calculated as follows:

$$G(q) = \sum_{i=1}^{4} (P.E_i)$$

```
%% 5. Gravity Matrix G(q)
disp('Calculating Gravity Matrix...');
g_vec = [0; 0; -9.81]; % Gravity acceleration along -Z
Pot = sym(0);
% Potential Energy = sum(m * g * h)
for i = 1:4
    Pot = Pot - masses(i) * g_vec.' * P{i};
end
% Gravity vector is the gradient of Potential Energy wrt q
G_sym = simplify(jacobian(Pot, q).');
```

Figure 42: Calculating Gravity Matrix as follows

Finally, a **Coriolis Matrix** is calculated as follows using Christoffel Symbols:

$$c_{ijk} = \frac{1}{2}\left(\frac{\partial m_{ij}}{\partial q_k} + \frac{\partial m_{ik}}{\partial q_j} + \frac{\partial m_{jk}}{\partial q_i}\right)$$

$$C_{ij}(q, \dot{q}) = \sum_{i=0}^{4} c_{ijk}\, \dot{q}_k$$

```
86  %% 6. Coriolis Matrix C(q, dq)
87  disp('Calculating Coriolis Matrix...');
88  C_sym = sym(zeros(4, 4));
89  % Computed analytically using Christoffel symbols
90  for k = 1:4
91      for j = 1:4
92          for i = 1:4
93              c_ijk = 0.5 * (diff(M_sym(k, j), q(i)) + diff(M_sym(k, i), q(j)) - diff(M_sym(i, j), q(k)));
94              C_sym(k, j) = C_sym(k, j) + c_ijk * dq(i);
95          end
96      end
97  end
98  C_sym = simplify(C_sym);
```

Figure 43: Getting the Coriolis Matrix

```
99  %% 7. Evaluate and Print Numerical Matrices
100 disp('========================================');
101 disp('          NUMERICAL RESULTS             ');
102 disp('========================================');
103
104 q_val = thetas.';
105 dq_val = theta_dot.';
106
107 % Evaluate M(q)
108 M_num = double(subs(M_sym, q, q_val));
109 disp('Inertia Matrix M(q):');
110 disp(M_num);
111 disp(' ');
112
113 % Evaluate C(q, dq)
114 C_num = double(subs(C_sym, [q; dq], [q_val; dq_val]));
115 disp('Coriolis Matrix C(q, dq):');
116 disp(C_num);
117 disp(' ');
118
119 % Evaluate G(q)
120 G_num = double(subs(G_sym, q, q_val));
121 disp('Gravity Matrix G(q):');
122 disp(G_num);
```

```
========================================
          NUMERICAL RESULTS
========================================
Inertia Matrix M(q):
    0.0289   -0.0000   -0.0000   -0.0000
   -0.0000    0.0366    0.0214    0.0088
   -0.0000    0.0214    0.0165    0.0071
   -0.0000    0.0088    0.0071    0.0038


Coriolis Matrix C(q, dq):
   -0.0025   -0.0013   -0.0000   -0.0000
    0.0013   -0.0012    0.0005         0
    0.0000   -0.0017         0         0
    0.0000   -0.0006         0         0


Gravity Matrix G(q):
         0
    1.0314
    0.6875
    0.2481
```

Figure 44: Getting the results for the given Joint Angles

Finally, **Joints Torque** are calculated as follows

```
123 %% 8. Calculate Joint Torques (Tau)
124 disp('Calculating Joint Torques...');
125 theta_ddot = [0.05, -0.1, 0.05, 0.0];
126 q_ddot_val = theta_ddot.';
127 Tau_num = M_num * q_ddot_val + C_num * dq_val + G_num;
128 disp('Joint Torques Tau (Nm):');
129 disp(Tau_num);
```

```
Calculating Joint Torques...
Joint Torques Tau (Nm):
    0.0009
    1.0287
    0.6859
    0.2474
```

Figure 45: Getting the Joints' Torques

6.2. Using Robotic Systems Toolbox:

The following procedure is carried out to compute Robot's Dynamics using Robotic Systems Toolbox

```
%% 1. Assign Dynamic Properties to SupernovaArm
SupernovaArm.DataFormat='column';
%% 2. Assign Dynamics
SupernovaArm.Gravity = [0 0 -9.81];
q = [0; -pi/6; -pi/3; 0];
qd = [0.1; 0.2; -0.1; -0.1];
qdd = [0.05; -0.1; 0.05; 0.0];
%% 3. Getting Dynamic Matrices
M = massMatrix(SupernovaArm,q);
Cv = velocityProduct(SupernovaArm,q,qd);
G = gravityTorque(SupernovaArm,q);
Tau = inverseDynamics(SupernovaArm,q,qd,qdd);
Tau_check = M*qdd + Cv + G;
%% 4. Results
disp('Inertia Matrix M(q)');
disp('=====================================');
disp(M);
disp('Coriolis/Centrifugal Vector');
disp('=====================================');
disp(Cv);
disp('Gravity Vector');
disp('=====================================');
disp(G);
disp(' ');
disp('Required Joint Torque');
disp('=====================================');
disp(Tau);
disp('Checked Joint Torque');
disp('=====================================');
disp(Tau_check);
disp('Error');
disp('=====================================');
disp(Tau_check-Tau);
```

Figure 46: Getting Robot's Dynamics using Robotic Systems Toolbox

The result of the script is displayed as follows:

```
Inertia Matrix M(q)
=====================================
    0.0165    0.0004    0.0000    0.0000
    0.0004    0.0220    0.0108    0.0028
    0.0000    0.0108    0.0076    0.0021
    0.0000    0.0028    0.0021    0.0008

Coriolis/Centrifugal Vector
=====================================
   1.0e-03 *

   -0.3420
   -0.0798
   -0.2104
   -0.0419

Gravity Vector
=====================================
    0.0000
    0.7260
    0.4359
    0.0952
```

```
Required Joint Torque
=====================================
    0.0004
    0.7243
    0.4350
    0.0949

Checked Joint Torque
=====================================
    0.0004
    0.7243
    0.4350
    0.0949

Error
=====================================
   1.0e-16 *

   -0.8338
         0
    0.5551
    0.1388
```

Figure 47: Results of Dynamic Analysis using Robotic Systems Toolbox

6.3. Using Simscape Multibodies Toolbox:

An Inverse Dynamics Block is used to compute the Required Joint Torques for the Manipulator at a given Joint Angles, speeds and Accelerations as well

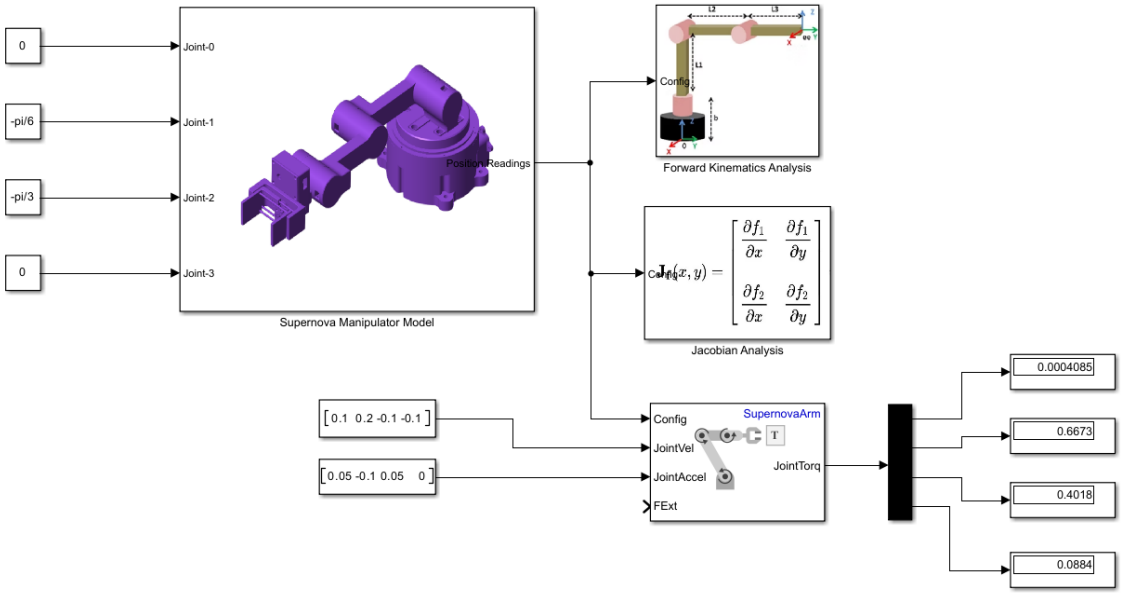


Figure 49: Inverse Dynamics to get the Required Joint Torques

Also Dynamics Matrices are carried out to compare between the Inverse Dynamics and The results of the Matrices summation as follows. It is deduced that the resulting Joint Torques are too close **"Acceptable"**

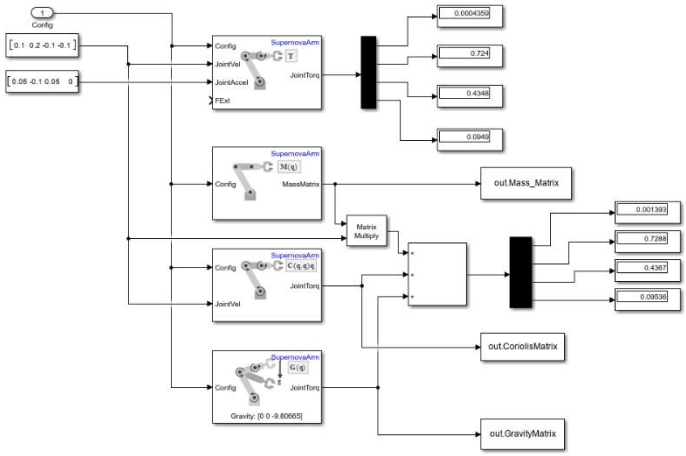


Figure 50: Getting the Joints' Torques using the Torque Matrices

```
>> MassMatrix

MassMatrix =

    0.0165    0.0004    0.0000    0.0000
    0.0004    0.0220    0.0108    0.0028
    0.0000    0.0108    0.0076    0.0021
    0.0000    0.0028    0.0021    0.0008

>> CoriolisVector

CoriolisVector =

   1.0e-03 *

   -0.3420
   -0.0798
   -0.2104
   -0.0419

>> GravityVector

GravityVector =

    0.0000
    0.7258
    0.4357
    0.0951
```

Figure 48: Mass, Coriolis and Gravity Matrices

6.4. Comparison between the three Methods

The following Comparison Table is carried out to compare between the Joint Torques required by the Manipulator using the three methods.

Table 6: Comparison Table Between the three method regarding inverse dynamics

POC	Analytical Method	Robotics System Toolbox	Simulink/ Simscape Method®
T_1	0.0009	0.0004	0.00041
T_2	1.0287	0.7243	0.7240
T_3	0.6859	0.4350	0.4348
T_4	0.2474	0.0949	0.0949

From the following we notice that there is differences between the three methods due to the following

1. The **Inertia is different at irregular shapes** on computing it using simscape multibodies comparing to the Analytical Methods
2. Approximations made about the Center Of Mass in analytical methods where it is assumed **at the joint itself** as most of the weight distribution is at the actuator itself, which is close to the joint to ease its calculation

7. Trajectory Planning:

7.1. Analytical Method:

The following Matlab Script is carried out as follows to calculate the motion profile using the quintic polynomials as it provides the least jerk onto the Arm's links

```
%% 1. Define the Extracted Waypoints Matrix (X, Y, Z rows)
Waypoints = [0.35690 0.28690 0.00025 0.00025 0.00025 0.28690 0.35690;
             0.00025 0.00025 0.28690 0.35690 0.28690 0.00025 0.00025;
             0.11730 0.23854 0.23854 0.11730 0.23853 0.23854 0.11730];
% Define the specific timestamps for each waypoint
t_waypoints = [0, 1, 3, 4, 5, 7, 8];
[num_axes, num_points] = size(Waypoints);
num_segments = num_points - 1;
% Timing Parameters
dt = 0.01;        % Resolution of the plot (time step)
time_total = [];
pos_total = [];
vel_total = [];
acc_total = [];
%% 2. Calculate Trajectories Segment by Segment
for i = 1:num_segments
    q0 = Waypoints(:, i);
    q1 = Waypoints(:, i+1);
    % Calculate the specific duration for this segment
    T_seg = t_waypoints(i+1) - t_waypoints(i);
    % Time vector for current segment
    t = 0:dt:T_seg;
    tau = t / T_seg; % Normalized time [0 to 1]
    % Preallocate segment matrices
    pos = zeros(num_axes, length(t));
    vel = zeros(num_axes, length(t));
    acc = zeros(num_axes, length(t));
    % Apply Quintic Equations for X, Y, and Z
    for j = 1:num_axes
        dq = q1(j) - q0(j);
        % Calculate Position, Velocity, and Acceleration
        pos(j, :) = q0(j) + dq * (10*tau.^3 - 15*tau.^4 + 6*tau.^5);
        vel(j, :) = (dq / T_seg) * (30*tau.^2 - 60*tau.^3 + 30*tau.^4);
        acc(j, :) = (dq / T_seg^2) * (60*tau - 180*tau.^2 + 120*tau.^3);
    end
    % Append to the total arrays
    % Shift the time vector by the absolute start time of the segment
    time_total = [time_total, t_waypoints(i) + t(1:end-1)];
    pos_total = [pos_total, pos(:, 1:end-1)];
    vel_total = [vel_total, vel(:, 1:end-1)];
    acc_total = [acc_total, acc(:, 1:end-1)];
end
% Add the very final point to cap off the arrays
time_total = [time_total, t_waypoints(end)];
pos_total  = [pos_total, Waypoints(:, end)];
vel_total  = [vel_total, zeros(num_axes, 1)];
acc_total  = [acc_total, zeros(num_axes, 1)];
```

Figure 51: Matlab Script for Trajectory Planning using Robotic Systems Toolbox

Where the results are Visualized As follows:

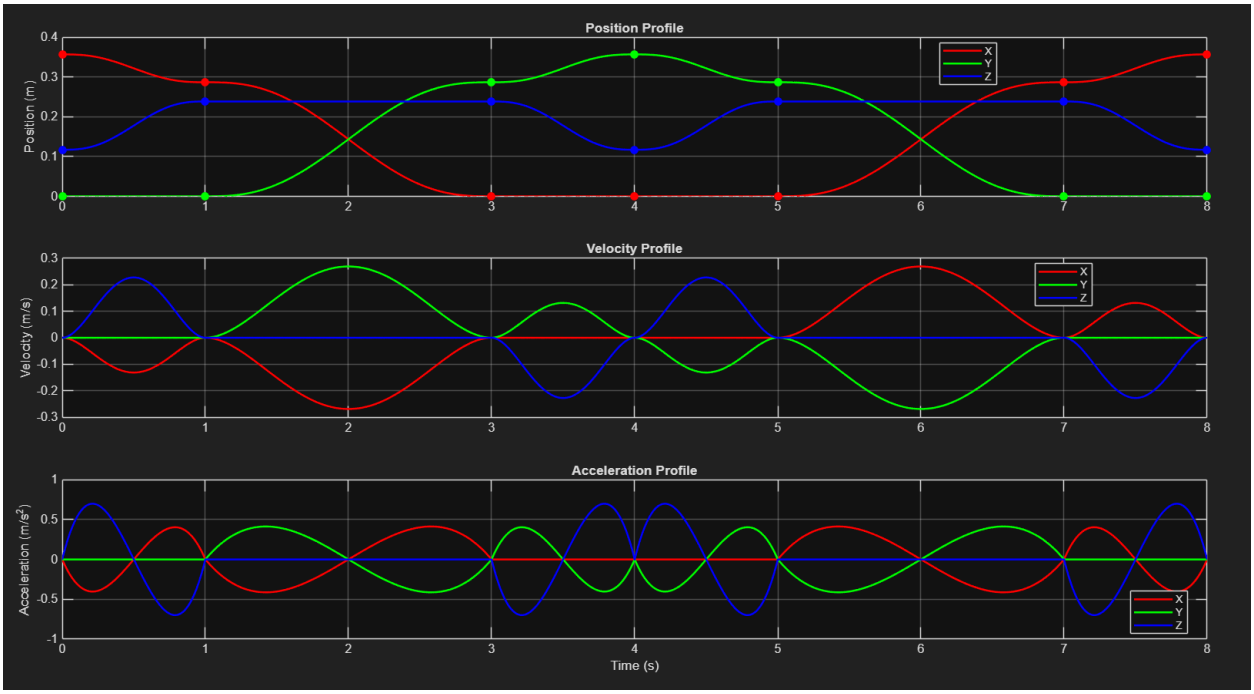


Figure 52: The motion Profiles of the given waypoints

7.2. Using Robotic Systems Toolbox:

The following script is written where a quinticpolytraj() function is used to perform the Trajectory Analysis

```
%% 1. Define waypoints (3x7) and corresponding timestamps
waypoints = [0.35690 0.28690 0.00025 0.00025 0.00025 0.28690 0.35690;
             0.00025 0.00025 0.28690 0.35690 0.28690 0.00025 0.00025;
             0.11730 0.23854 0.23854 0.11730 0.23853 0.23854 0.11730];
timePoints = [0, 1, 3, 4, 5, 7, 8];
tVec = 0:0.001:8;
%% 2. Generate quintic polynomial trajectory
[q, qd, qdd] = quinticpolytraj(waypoints, timePoints, tVec);
```

Figure 53: Performing Trajectory Analysis using Robotic Systems Toolbox

Then the final results are plotted as follows:

```
%% 3. Plot results
figure('Name', 'Quintic Polynomial Trajectory using Robotics System Toolbox');
% Position
subplot(3,1,1);
plot(tVec, q(1,:), 'b-', 'LineWidth', 1.5); hold on;
plot(tVec, q(2,:), 'r-', 'LineWidth', 1.5);
plot(tVec, q(3,:), 'g-', 'LineWidth', 1.5);
plot(timePoints, waypoints(1,:), 'bo', 'MarkerSize', 6, 'MarkerFaceColor', 'b');
plot(timePoints, waypoints(2,:), 'ro', 'MarkerSize', 6, 'MarkerFaceColor', 'r');
plot(timePoints, waypoints(3,:), 'go', 'MarkerSize', 6, 'MarkerFaceColor', 'g');
xlabel('Time (s)'); ylabel('Position');
title('Position vs Time');
legend('x', 'y', 'z', 'Location', 'best');
grid on;
% Velocity
subplot(3,1,2);
plot(tVec, qd(1,:), 'b-', 'LineWidth', 1.5); hold on;
plot(tVec, qd(2,:), 'r-', 'LineWidth', 1.5);
plot(tVec, qd(3,:), 'g-', 'LineWidth', 1.5);
xlabel('Time (s)'); ylabel('Velocity');
title('Velocity vs Time (zero at all waypoints)');
legend('x', 'y', 'z', 'Location', 'best');
grid on;
% Acceleration
subplot(3,1,3);
plot(tVec, qdd(1,:), 'b-', 'LineWidth', 1.5); hold on;
plot(tVec, qdd(2,:), 'r-', 'LineWidth', 1.5);
plot(tVec, qdd(3,:), 'g-', 'LineWidth', 1.5);
xlabel('Time (s)'); ylabel('Acceleration');
title('Acceleration vs Time (zero at all waypoints)');
legend('x', 'y', 'z', 'Location', 'best');
grid on;
```

Figure 54: Plotting the Motion Profiles of Quintic Polynomial Motion Profiles

The Result can be visualized as follows:

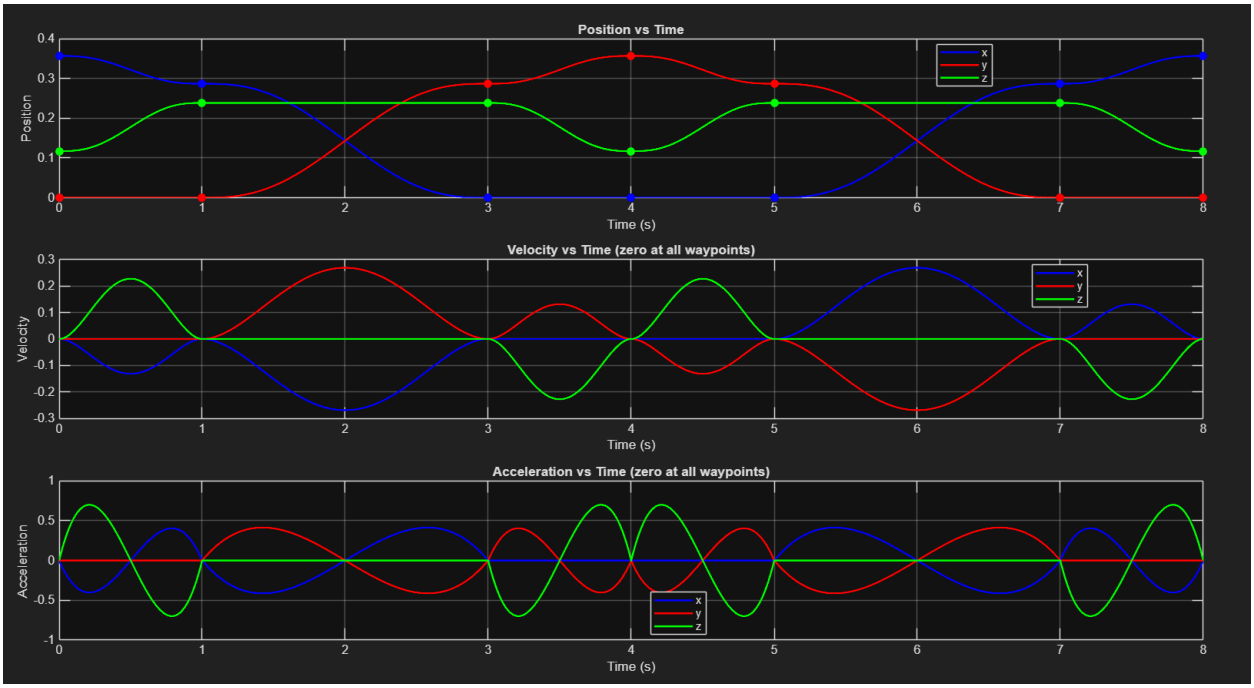


Figure 55: Motion Profile Graphs using Quintic Method

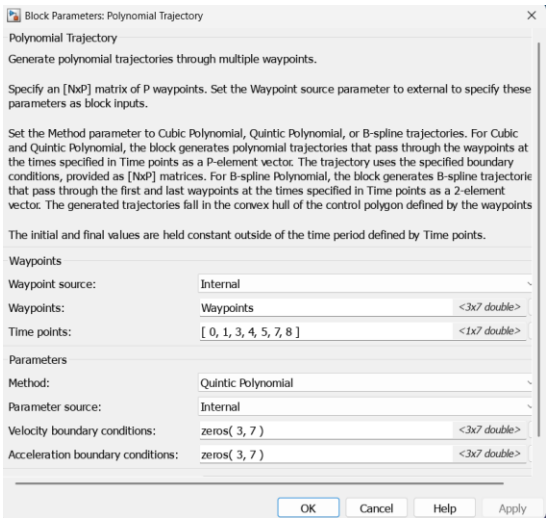
7.3. Using Simscape Multibodies Method:

The polynomial Trajectory planning is carried for predefined Waypoints taken to perform Pick-And-Place Operations. The waypoints are given as follow

```
Waypoints=[0.35690 0.28690 0.00025 0.00025 0.00025 0.28690 0.35690;
           0.00025 0.00025 0.28690 0.35690 0.28690 0.00025 0.00025;
           0.11730 0.23854 0.23854 0.11730 0.23853 0.23854 0.11730];
```

Figure 56: Waypoints given for trajectory planning

Then a Quintic Polynomial function is carried out using the **polynomial trajectory** block, where the **waypoints and the timestamps** are set as follows. Then the position is fed through the **inverse kinematics subsystem**



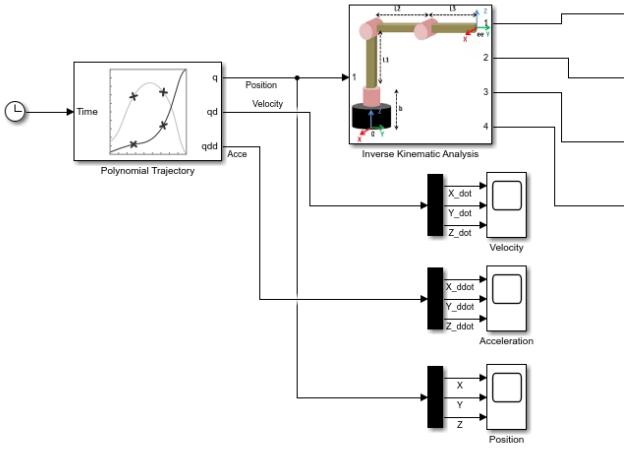


Figure 58: Setting the Quintic Polynomial Parameters

then the **initial guess** is updated continuously to avoid abrupt changes in joint positions as follows

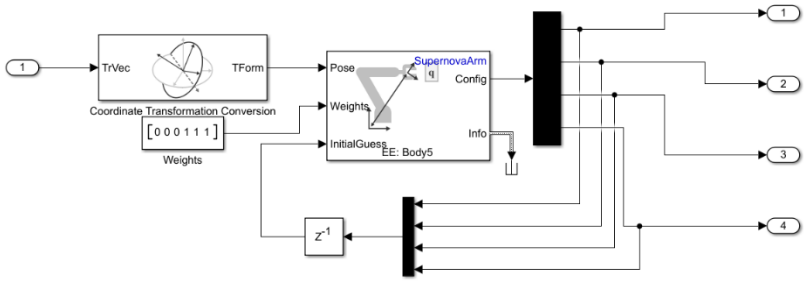


Figure 57: Adjusting the Initial Guess of the Inverse Kinematics in Trajectory planning

Then the Motion Profile is plotted as follows:

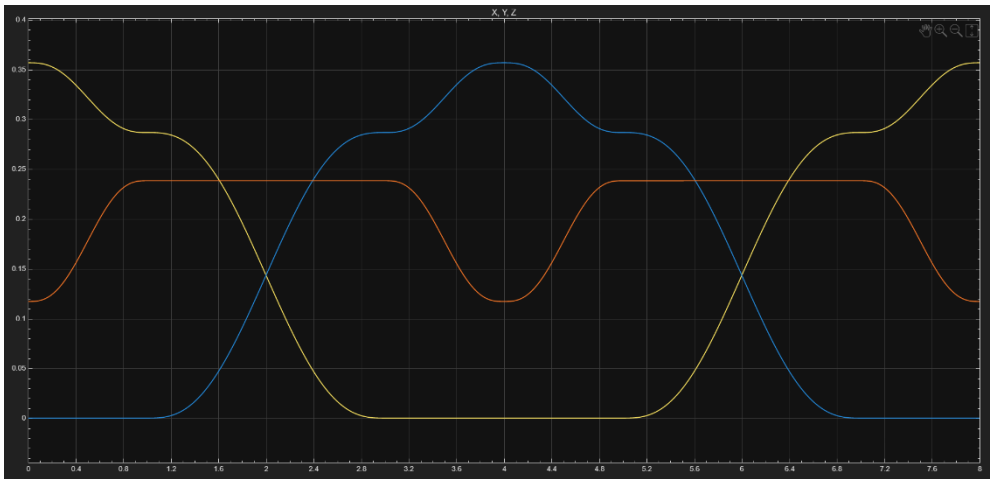


Figure 59: Position Profile

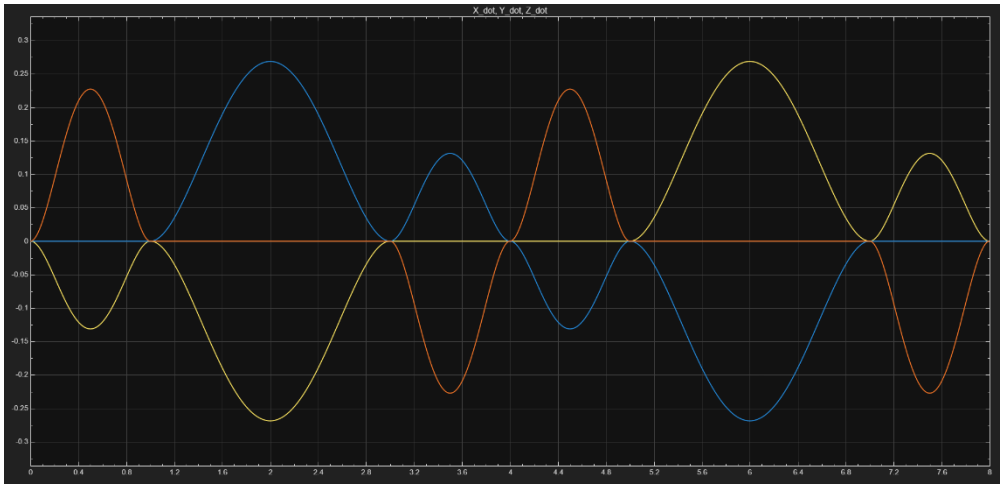


Figure 61: Velocity Profile

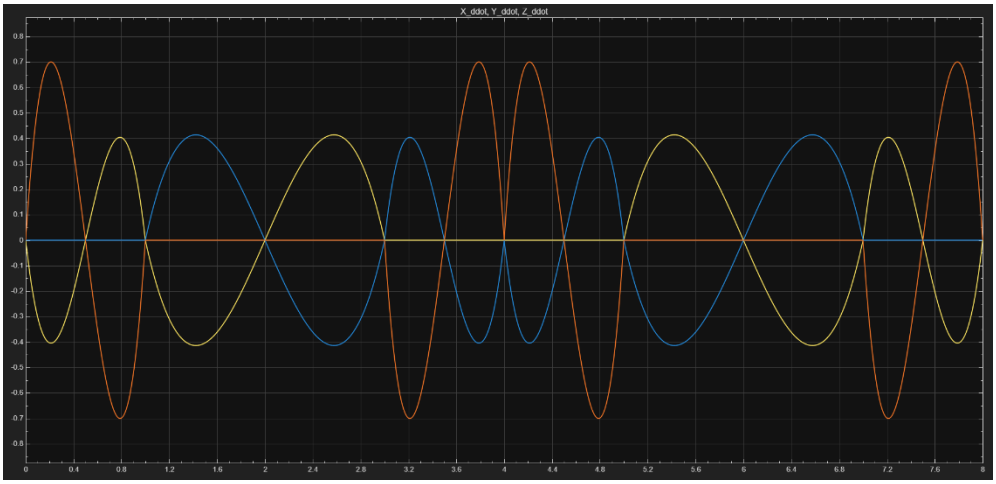


Figure 60: Acceleration Profile

7.4. Comparison Between the Three Methods:

A comparison Table is carried out to compare between the trajectories using the three different methods

Table 7: Position Profile Matrix Calculations

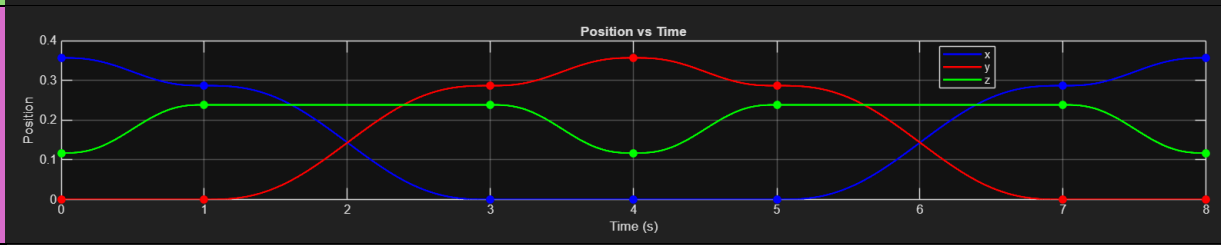
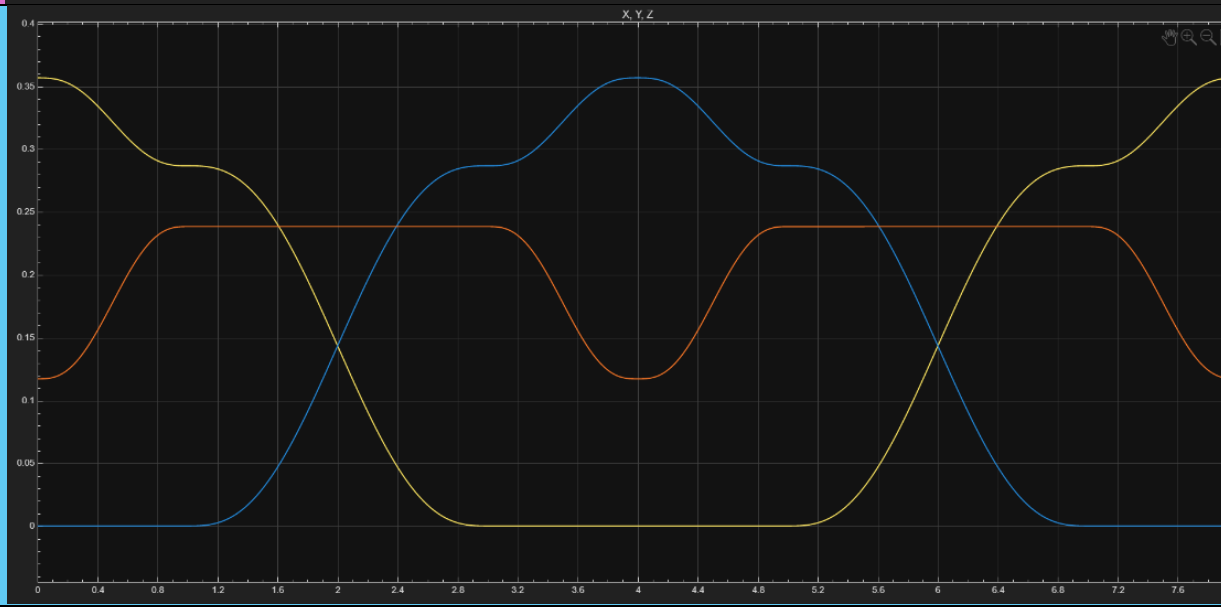
Position Profile Comparison	
Analytical Method	
Robotic Systems Toolbox	
Simscape Multibodies Method®	

Table 8: Velocity Profile Comparison

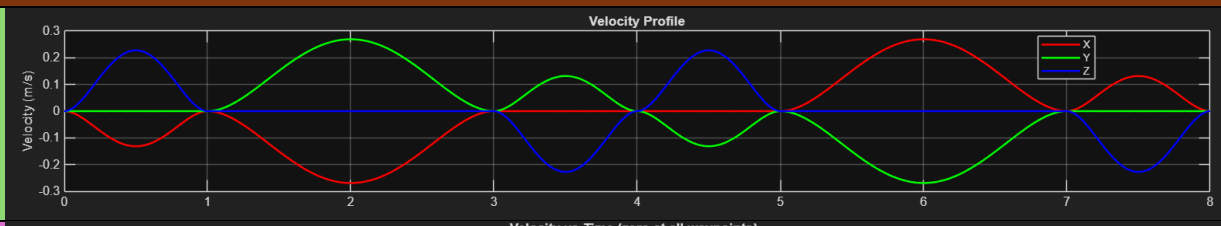
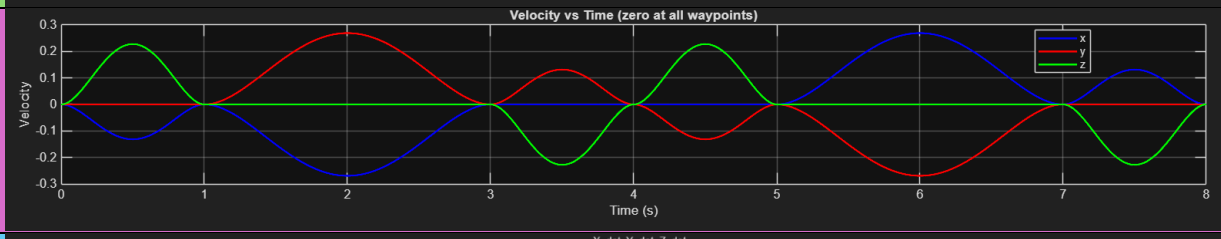
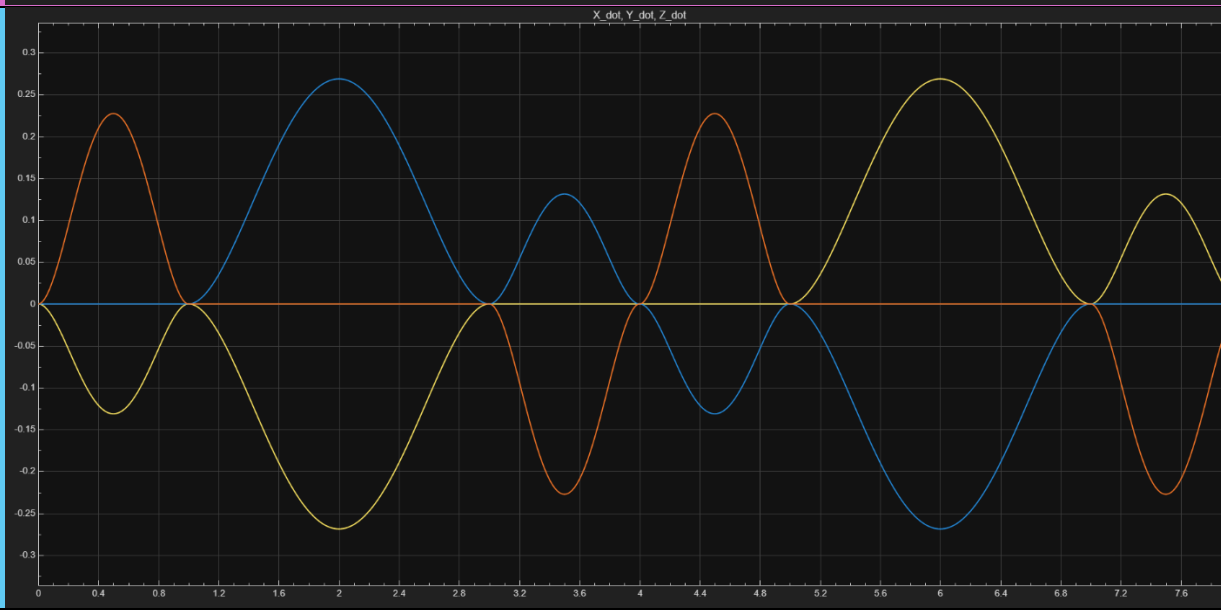
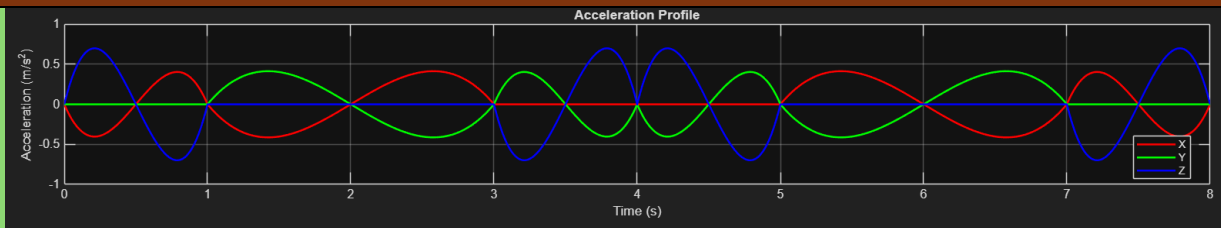
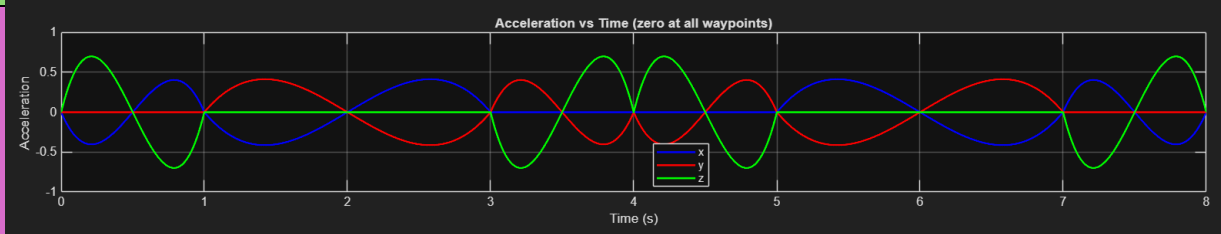
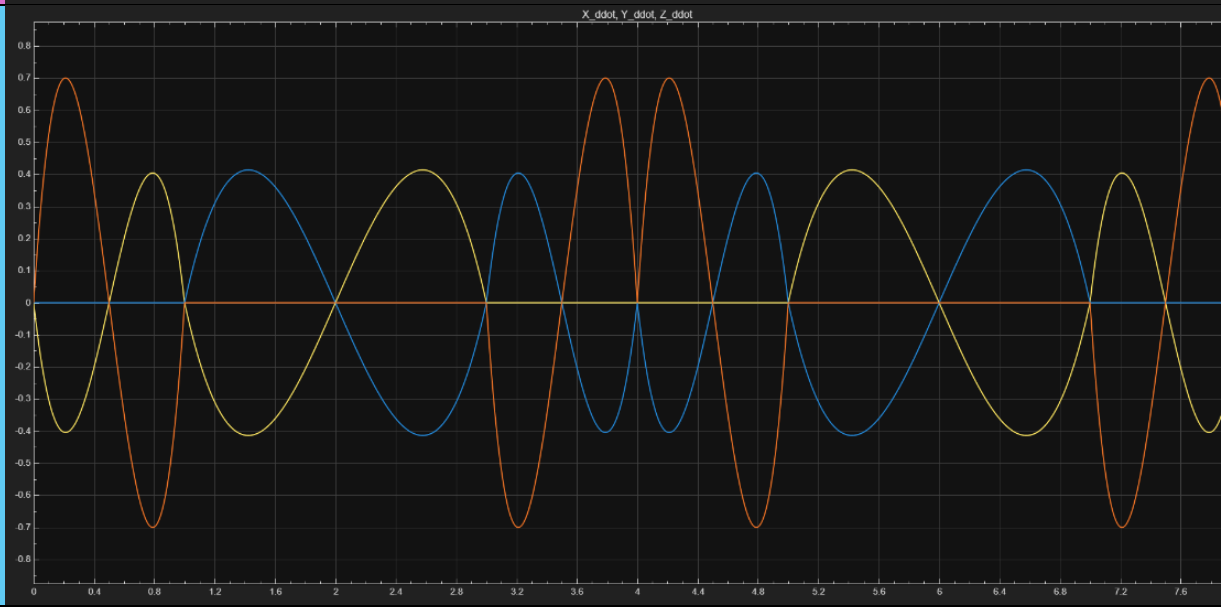
Velocity Profile Comparison	
Analytical Method	
Robotic Systems Toolbox	
Simscape Multibodies Method®	

Table 9: Acceleration Profile Comparison

Acceleration Profile Comparison	
Analytical Method	
Robotic Systems Toolbox	
Simscape Multibodies Method®	

From the following we can see that all the graphs are similar to each other, hence the Analysis are **valid**

8. Torque Control:

8.1. Using Robotic Systems Toolbox:

The following script used to solve the **PD control** of the Manipulator where the desired angle is **1 rad** for each joint and the joints are **initially at angle equals to zero**. Where the solver continuously computes the **Inertia Matrices, Coriolis Matrices and Gravity Matrices** to get the joint angles in a realtime way

```
%% 1. Load the Rigid Body Tree Model
numJoints = SupernovaArm.NumBodies - 1;
Kp = 900;
Kd = 200;
% Time Parameters
dt = 0.01;
t_end = 2.0;
time = 0:dt:t_end;
% Desired constant state
q_desired   = ones(numJoints, 1); % 1 rad for each joint
dq_desired  = zeros(numJoints, 1);
ddq_desired = zeros(numJoints, 1);
% Initial state
x0 = [zeros(numJoints,1); zeros(numJoints,1)];  % [q; dq]
%% 2. Define the ODE function (derivative of state)
odefun = @(t, x) robotDynamics(t, x, SupernovaArm, q_desired, dq_desired, ddq_desired, Kp, Kd);
% Solve using ode15s (stiff solver, like Simscape)
options = odeset('OutputFcn', @odeplot);  % optional
[t_out, x_out] = ode15s(odefun, time, x0, options);
% Extract q and dq from solution
q_meas_history = x_out(:, 1:numJoints)';
dq_meas_history = x_out(:, numJoints+1:end)';
% q_desired repeated for plotting
q_des_history = repmat(q_desired, 1, length(t_out));
```

Figure 62: Solving the position control based on torque control using Robotic Systems Toolbox

Where it is visualized as follows:

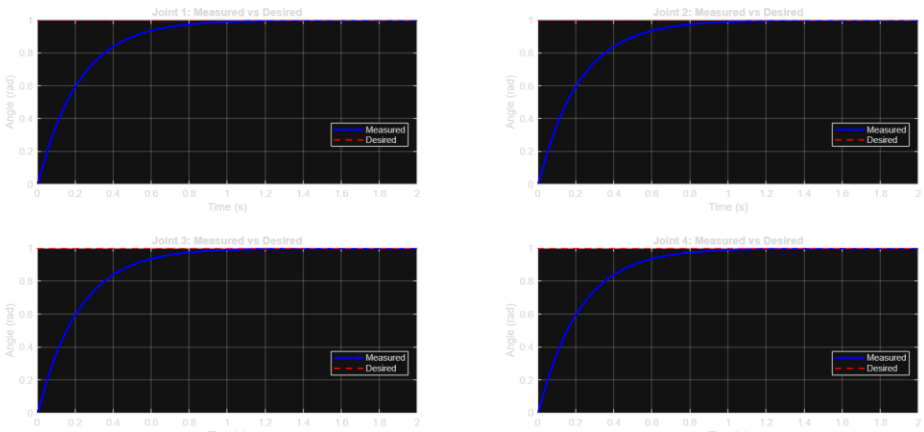


Figure 63: Plots of the Joints response on controlling it

8.2. Using Simscape Multibodies Method:

Using the following Control loop

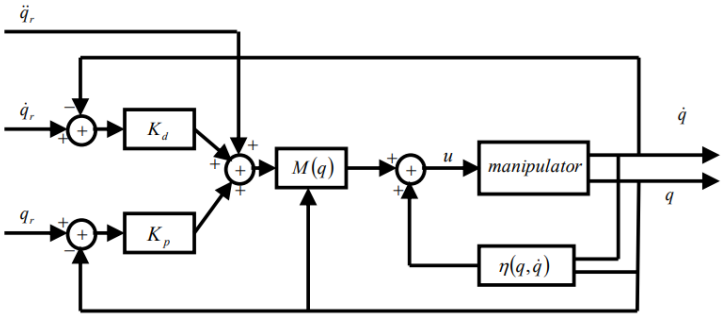


Figure 64: Position Control loop through controlling torque

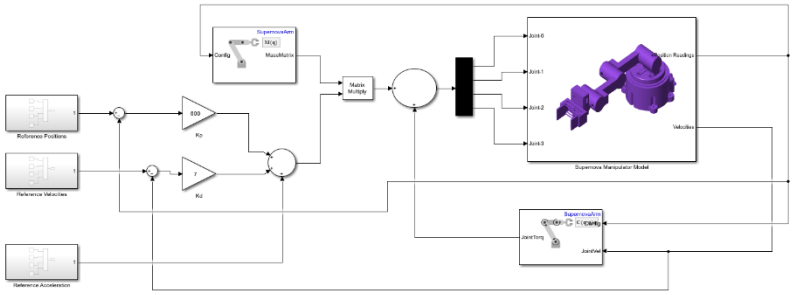


Figure 65: Control Loop implementation in Simscape

Then PD Tuning is taking space until it reaches steady space:

1. K_P= 600, K_D = 7

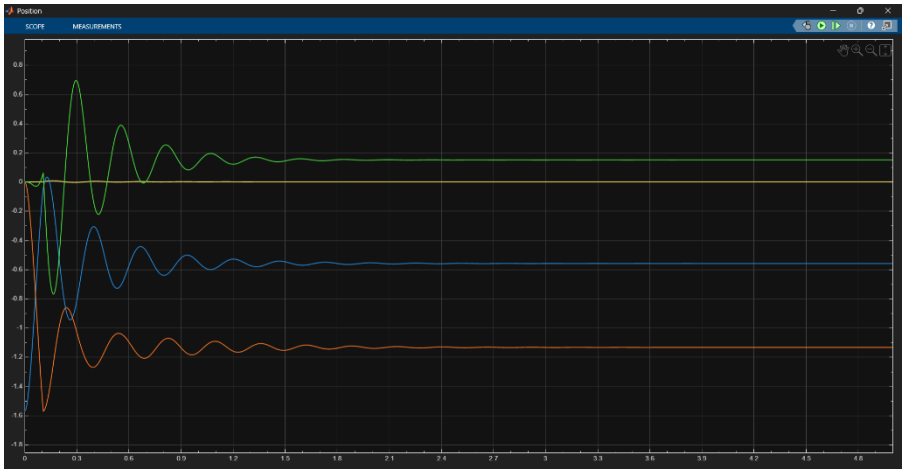


Figure 66: Trial 01 PD Tuning

2. K_p=600, K_D = 350

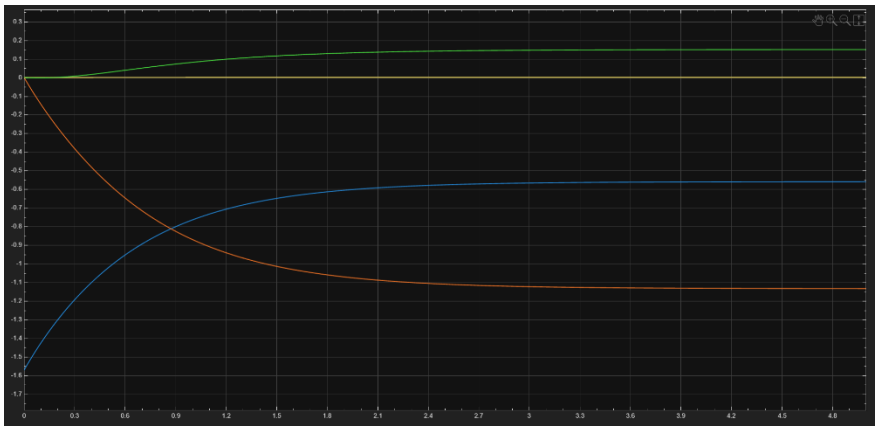


Figure 67: Trial 02: PD Tuning

3. K_p= 900, K_D = 200

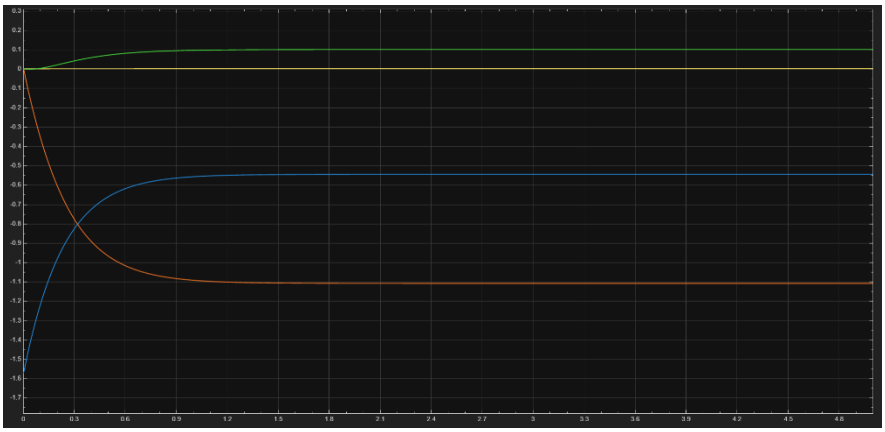
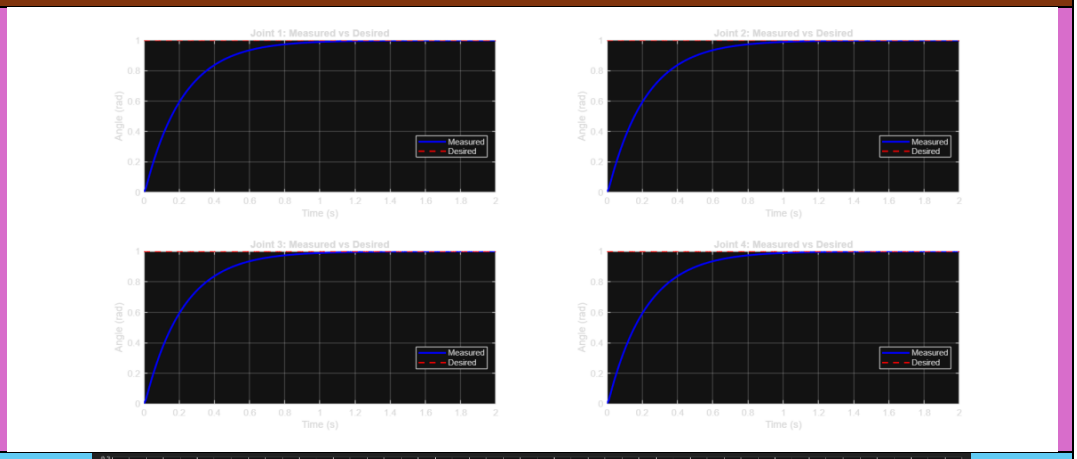
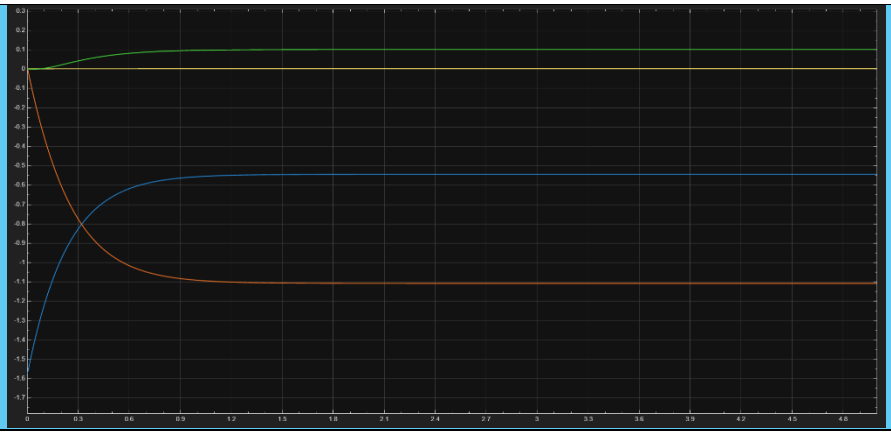


Figure 68 : Trial 03: PD Tuning

8.3. Comparison Between the two Methods:

The following Comparison is carried out to compare between the angle's responses using the two different methods for the **same K_D and K_p:**

Table 10: Comparison Between the responses of the Joint Angles using 2 different methods

Response Comparison	
Robotic Systems Toolbox	
Simscape Multibodies Method®	

As observable from the table, all joints reach at the steady state at almost 1 second, hence the calculations are **valid**.

9. References:

All Matlab files, Simulink Models and Videos are pushed through our GitHub Repository

https://github.com/philohani/Robotics_Project.git